Veri Görselleştirme: ggplot2 Ders Notları (1–4. Haftalar)

Dr. Öğr. Üyesi Zeynep ATLI

2026

Contents

1 Derse Başlarken **3**

1.1 Dersin Amacı . . . 3

1.2 Uygulama Süreci . . . 3

1.3 R Kodunu Okumaya Başlarken . . . 3

2 1. Veri Setini Tanıyalım **3**

2.1 1.1. `head()` – İlk gözlemleri görüntüleme . . . 4

2.2 1.2. `dim()` – Veri setinin boyutları . . . 4

2.3 1.3. `names()` – Değişken adları . . . 5

2.4 1.4. `str()` – Veri setinin yapısı . . . 5

2.5 1.5. `summary()` – Özet istatistikler . . . 5

2.6 1.6. `table()` – Grupları tanıma . . . 6

2.7 1.7. Kullanacağımız değişkenler . . . 6

2.8 1.8. Fonksiyonları karşılaştıralım . . . 7

2.9 1.9. Uygulama . . . 7

3 2. ggplot2 Kodunun İskeleti **7**

3.1 2.1. Kodun adım adım kurulması . . . 8

4 3. Aesthetics: Görsel Özellikler **11**

4.1 3.1. Mapping: Özelliği bir değişkene bağlama . . . 11

4.2 3.2. Setting: Sabit bir görünüm verme . . . 12

4.3 3.3. Karşılaştırma: `aes()` içinde ve dışında . . . 13

4.4 3.4. Global ve yerel eşleştirme . . . 15

5 4. Kategorik Değişken ve `factor()` **17**

6 5. `color`, `fill`, `shape`, `size`, `alpha` **19**

6.0.1 `fill` ve `color` Arasındaki Fark . . . 20

7 6. `geom_bar()`: Sütun Grafiği **22**

7.1 6.1. Sütunların üzerine sayıları yazmak: `geom_text()` . . . 23

7.1.1 `geom_bar()` ve `geom_col()` Arasındaki Fark . . . 24

8 7. `geom_histogram()`: Histogram **27**

8.1 7.1. `bins` ne demek? . . . 27

8.2 7.2. `binwidth` nedir? . . . 28

8.3 7.3. Histogramın yorumlanması . . . 29

9 8. `geom_density()`: Yoğunluk Eğrisi **29**

10 9. `geom_boxplot()`: Kutu Grafiği **30**

10.1 9.1. Çeyrekler ve sınırlar nasıl hesaplanır? . . . 31

10.2 9.2. Gerçek veriyle grup karşılaştırması . . . 33

10.3 9.3 Kutu Grafiği Üzerine Gözlemleri Eklemek: `geom_jitter()` . . . 34

11 10. `geom_violin()`: Keman Grafiği **36**

11.1 10.1. Violin ile boxplot birlikte . . . 37

11.1.1 `trim` Argümanı Nedir? . . . 38

12 11. Hangi Grafik Ne Zaman? **40**

13 Scales (Ölçekler): Veri Değerleri Grafikte Nasıl Gösterilir? **41**

14 13. `labs()`: Başlık ve Eksenler **43**

15 14. `theme()`: Grafiğin Görünümü **43**

16 15. Tam Bir ggplot2 Örneği **45**

17 17. Sık Yapılan Hatalar **46**

17.1 Hata 1: Sayısal `cyl` ile gruplama beklemek . . . 46

17.2 Hata 2: `fill = cyl` ifadesini `aes()` dışında yazmak . . . 47

17.3 Hata 3: Histogramda `bins = 5` ifadesini . . . 48

17.4 Hata 4: Boxplot çizgisini teorik sınırla karıştırmak . . . 48

17.5 Hata 5: Yoğunluk yüksekliğini frekans sanmak . . . 48

18 18. Öğrenci Uygulamaları **48**

19 19. Kendinizi Kontrol Edin **49**

20 20. Ders Sonu Özeti **49**

1 Derse Başlarken

1.1 Dersin Amacı

Bu ders materyali, öğrencilerin `ggplot2` paketinin temel yapısını ve veri görselleştirme sürecinde kullanılan fonksiyonların işlevlerini kavramalarını desteklemek amacıyla hazırlanmıştır. İçerik, kodların yalnızca uygulanmasına değil, her bir fonksiyonun, argümanın ve görsel eşleştirmenin grafik üzerindeki etkisinin anlaşılmasına odaklanmaktadır. Konular, temel örneklerden başlayarak aşamalı biçimde ilerlemekte ve uygulamalarla pekiştirilmektedir.

1.2 Uygulama Süreci

Öğrencilerin her uygulama öncesinde ilgili kodu incelemeleri, kodun oluşturacağı grafik veya çıktıya ilişkin tahminde bulunmaları ve ardından RStudio ortamında çalıştırarak sonuçları değerlendirmeleri beklenmektedir. Kod üzerinde farklı değişkenler ve parametreler kullanarak değişiklikler yapmaları, bu değişikliklerin sonuçlara etkilerini gözlemlemeleri ve elde edilen bulguları yorumlamaları önerilmektedir.

1.3 R Kodunu Okumaya Başlarken

R'de bir **fonksiyon**, belirli bir işlemi gerçekleştirmek üzere kullanılan komuttur. Fonksiyon adından sonra gelen parantez `()` içine, gerekiyorsa **argümanlar** yazılır. Argümanlar, fonksiyonun hangi veri üzerinde veya hangi ayarlarla çalışacağını belirler.

```
# head() bir fonksiyondur.
# df: üzerinde işlem yapılan veri setidir.
# n = 3: kaç satır görüntüleneceğini belirleyen argümandır.
head(df, n = 3)
#>                mpg cyl disp  hp drat    wt  qsec vs am gear carb
#> Mazda RX4     21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
#> Mazda RX4 Wag 21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
#> Datsun 710    22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
```

- `#` işareti açıklama satırı başlatır; R bu satırı kod olarak çalıştırmaz.
- `<-` işareti sağdaki değeri soldaki nesneye atar: `df <- mtcars`.
- `$` işareti veri setindeki bir sütuna erişir: `df$mpg`.
- `=` işareti fonksiyon argümanlarına değer vermek için kullanılır: `n = 3`.
- R, büyük/küçük harfe duyarlıdır: `mpg` ile `MPG` aynı ad değildir.

2 1. Veri Setini Tanıyalım

Veri görselleştirme sürecinde ilk adım, veri setinin yapısını incelemektir. Hangi değişkenlerin bulunduğunu, bunların nasıl saklandığını ve değerlerin nasıl dağıldığını bilmek, uygun grafik türünü ve doğru görsel eşleştirmeyi seçmeyi kolaylaştırır.

R ile birlikte gelen `mtcars` veri seti **32 otomobile ait 11 değişken** içerir. Bu derste `df` adlı bir nesne oluşturarak aynı veri setini kullanacağız.

```
# mtcars veri setini df adlı nesneye atıyoruz.
df <- mtcars
```

2.1 1.1. `head()` – İlk gözlemleri görüntüleme

`head()` fonksiyonu bir veri setinin ilk satırlarını gösterir. Varsayılan olarak **ilk 6 satır** görüntülenir. İkinci argüman olan `n` ile bu sayı değiştirilebilir.

```
# İlk 6 gözlemi görüntüler.
head(df)
#>                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
#> Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
#> Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
#> Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
#> Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
#> Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
#> Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1

# İlk 10 gözlemi görüntüler.
head(df, n = 10)
#>                   mpg cyl  disp  hp drat    wt  qsec vs am gear carb
#> Mazda RX4         21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
#> Mazda RX4 Wag     21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
#> Datsun 710        22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
#> Hornet 4 Drive    21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
#> Hornet Sportabout 18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
#> Valiant           18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
#> Duster 360        14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
#> Merc 240D         24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
#> Merc 230          22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
#> Merc 280          19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
```

Neden kullanıyoruz? Veri setindeki sütunların nasıl düzenlendiğini ve değişkenlerde hangi tür değerlerin bulunduğunu ilk bakışta kontrol etmek için. Ancak ilk satırlar tüm veri setini temsil etmek zorunda değildir.

Düşünme sorusu: `head(df, n = 3)` yazdığımızda kaç satır görmeyi bekleriz? Sütun sayısı değişir mi?

2.2 1.2. `dim()` – Veri setinin boyutları

`dim()` fonksiyonu satır ve sütun sayısını sırasıyla döndürür.

```
# İlk sayı satır (gözlem), ikinci sayı sütun (değişken) sayısıdır.
dim(df)
#> [1] 32 11
```

`mtcars` için sonuç `32 11` olur: 32 gözlem ve 11 değişken. Buradaki `[1]` ifadesi, R konsolunda gösterilen çıktının ilk elemanının konumunu belirtir; veri setindeki satır sayısının bir parçası değildir.

```
# Sadece satır sayısı
nrow(df)
#> [1] 32

# Sadece sütun sayısı
ncol(df)
#> [1] 11
```

Soru: `dim(df)` çıktısı `32 11` ise 11 otomobil mi, 11 değişken mi vardır?

2.3 1.3. `names()` – Değişken adları

`names()` fonksiyonu veri setindeki sütun adlarını listeler. Grafik kodunda kullanılacak değişken adlarını doğru yazmak için yararlıdır.

```
# Bütün sütunların adlarını gösterir.
names(df)
#>  [1] "mpg"  "cyl"  "disp" "hp"   "drat" "wt"   "qsec" "vs"   "am"   "gear"
#> [11] "carb"
```

Örneğin `mpg`, `disp`, `hp`, `wt` ve `cyl` bu veri setindeki değişken adlarıdır. `aes(x = disp, y = mpg)` yazarken bu adlardan yararlanırız.

2.4 1.4. `str()` – Veri setinin yapısı

`str()` (*structure*) fonksiyonu veri setinin yapısını, değişken adlarını, saklanma türlerini ve bazı örnek değerlerini gösterir.

```
# Değişken türlerini ve veri yapısını inceleyelim.
str(df)
#> 'data.frame':    32 obs. of  11 variables:
#>  $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
#>  $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
#>  $ disp: num  160 160 108 258 360 ...
#>  $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
#>  $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
#>  $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
#>  $ qsec: num  16.5 17 18.6 19.4 17 ...
#>  $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
#>  $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
#>  $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
#>  $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

`num` ifadesi sayısal veri türünü gösterir. `mtcars` veri setindeki değişkenler sayısal olarak saklanmaktadır. Bununla birlikte **saklanma türü**, değişkenin görselleştirmedeki rolünü tek başına belirlemez. Örneğin `cyl`, 4, 6 ve 8 silindirli otomobil **gruplarını** göstermek için `factor(cyl)` biçiminde kategorik olarak ele alınabilir.

Soru: `cyl` sayısal olarak saklanıyor diye onu her zaman sürekli bir değişken gibi mi göstermeliyiz?

2.5 1.5. `summary()` – Özet istatistikler

`summary()` fonksiyonu sayısal değişkenler için minimum, birinci çeyrek (Q1), medyan, ortalama, üçüncü çeyrek (Q3) ve maksimum gibi özet değerleri gösterir.

```
# Veri setindeki bütün değişkenlerin özetleri
summary(df)
#>       mpg             cyl             disp             hp
#>  Min.   :10.40   Min.   :4.000   Min.   : 71.1   Min.   : 52.0
#>  1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8   1st Qu.: 96.5
#>  Median :19.20   Median :6.000   Median :196.3   Median :123.0
#>  Mean   :20.09   Mean   :6.188   Mean   :230.7   Mean   :146.7
#>  3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0   3rd Qu.:180.0
#>  Max.   :33.90   Max.   :8.000   Max.   :472.0   Max.   :335.0
#>       drat             wt             qsec             vs
#>  Min.   :2.760   Min.   :1.513   Min.   :14.50   Min.   :0.0000
```

```
#>  1st Qu.:3.080   1st Qu.:2.581   1st Qu.:16.89   1st Qu.:0.0000
#>  Median :3.695   Median :3.325   Median :17.71   Median :0.0000
#>  Mean   :3.597   Mean   :3.217   Mean   :17.85   Mean   :0.4375
#>  3rd Qu.:3.920   3rd Qu.:3.610   3rd Qu.:18.90   3rd Qu.:1.0000
#>  Max.   :4.930   Max.   :5.424   Max.   :22.90   Max.   :1.0000
#>        am              gear            carb
#>  Min.   :0.0000   Min.   :3.000   Min.   :1.000
#>  1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:2.000
#>  Median :0.0000   Median :4.000   Median :2.000
#>  Mean   :0.4062   Mean   :3.688   Mean   :2.812
#>  3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:4.000
#>  Max.   :1.0000   Max.   :5.000   Max.   :8.000

# Yalnızca mpg değişkeninin özeti
summary(df$mpg)
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>   10.40   15.43   19.20   20.09   22.80   33.90
```

Burada `$` işareti, `df` içindeki **`mpg` sütununa erişmemizi** sağlar. Örneğin `df$mpg`, 32 otomobilin yakıt verimliliği değerlerinden oluşan bir vektördür.

`summary()` çıktısında yer alan **Q1, medyan ve Q3** değerlerini ileride `geom_boxplot()` bölümünde yeniden kullanacağız. Çeyreklerin hesaplanmasında farklı yöntemler bulunduğundan, `summary()` ve `quantile()` çıktıları ile `geom_boxplot()` içindeki *hinge* değerleri bazı veri setlerinde küçük farklılıklar gösterebilir.

2.6 1.6. `table()` – Grupları tanıma

```
# Her silindir grubundaki otomobil sayısını hesaplar.
table(df$cyl)
#>
#>  4  6  8
#> 11  7 14
```

`table()` her değerin kaç kez görüldüğünü sayar. `mtcars` veri setinde 4 silindirli 11, 6 silindirli 7 ve 8 silindirli 14 otomobil vardır.

2.7 1.7. Kullanacağımız değişkenler

Değişken	Açıklama	Bu dersteki kullanım
`mpg`	Yakıt verimliliği (mil/galon)	Sayısal
`disp`	Motor hacmi (inç küp)	Sayısal
`hp`	Motor gücü (beygir gücü)	Sayısal
`wt`	Araç ağırlığı (1000 libre)	Sayısal
`cyl`	Silindir sayısı (4, 6, 8)	Gruplamada kategorik

```
# cyl sayısal saklanır; factor() onu kategorik gruplar olarak ele alır.
factor(df$cyl)
#>  [1] 6 6 4 6 8 6 8 4 4 6 6 8 8 8 8 8 8 4 4 4 4 8 8 8 8 4 4 4 8 6 8 4
#> Levels: 4 6 8
```

2.8 1.8. Fonksiyonları karşılaştıralım

Fonksiyon	Temel görevi	Yanıtladığı soru
`head()`	İlk satırları gösterir	Veriler nasıl görünüyor?
`dim()`	Satır ve sütun sayısını verir	Veri setinin boyutu nedir?
`nrow()` / `ncol()`	Satır / sütun sayısını ayrı ayrı verir	Kaç gözlem / değişken var?
`names()`	Sütun adlarını listeler	Hangi değişkenler var?
`str()`	Yapı ve saklanma türlerini gösterir	Değişkenler nasıl saklanıyor?
`summary()`	Özet istatistikleri verir	Değerler nasıl özetlenebilir?
`table()`	Değerlerin sıklığını hesaplar	Her grupta kaç gözlem var?

2.9 1.9. Uygulama

Aşağıdaki soruları **önce kodları kendiniz yazarak** yanıtlayın:

1. `mtcars` veri setinde kaç otomobil vardır?
2. Kaç değişken vardır?
3. İlk 4 otomobili nasıl görüntülersiniz?
4. `mpg` değişkeninin medyanını hangi fonksiyonla öğrenebilirsiniz?
5. `cyl` değişkeni: her grupta kaç otomobil bulunur?
6. `df$hp` ifadesinde `$` işaretinin görevi nedir?

```
# Kodlarınızı buraya yazın.
# head(df, n = ...)
# dim(df)
# summary(df$...)
# table(df$...)
```

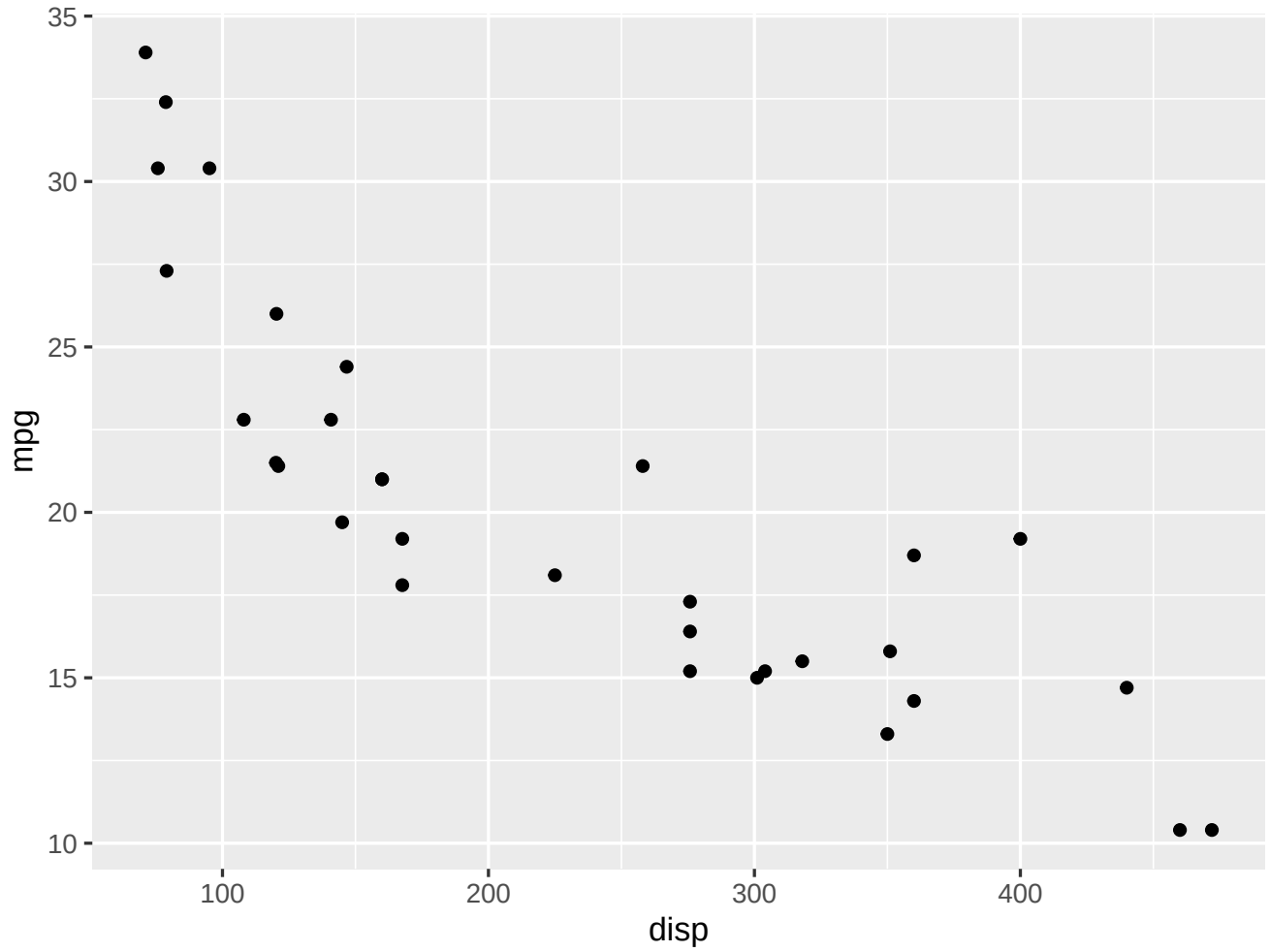
Temel yaklaşım: Veriyi tanı → değişkenleri ve veri türlerini değerlendir → araştırma sorusunu belirle → uygun grafiği seç → grafiği oluştur ve yorumla.

3 2. ggplot2 Kodunun İskeleti

Bir ggplot2 grafiğinin temel bileşenleri:

1. **Data:** Hangi veri seti?
2. **Mapping (`aes`):** Hangi değişken hangi görsel özelliğe bağlanacak?
3. **Geometry (`geom_*`):** Veriyi nokta, sütun, çizgi vb. hangi biçimde göstereceğiz?

```
ggplot(data = df,
       mapping = aes(x = disp, y = mpg)) +
  geom_point()
```



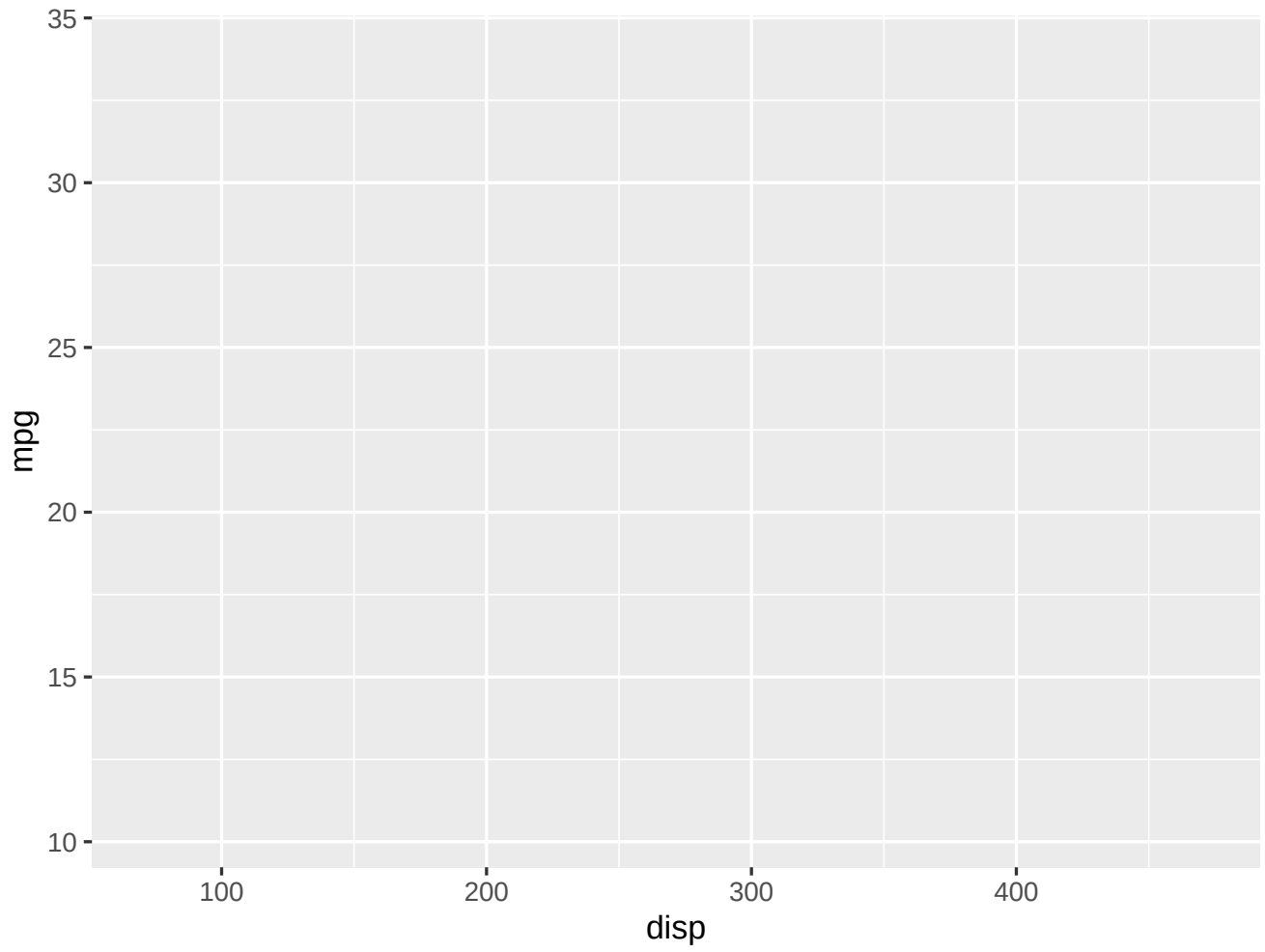
Kod	Ne anlama gelir?
`ggplot()`	Grafik nesnesini oluşturur.
`data = df`	Veriyi `df` nesnesinden alır.
`mapping = aes(...)`	Değişkenlerin görsel eşleştirmesini tanımlar.
`x = disp`	Yatay eksene motor hacmini yerleştirir.
`y = mpg`	Dikey eksene yakıt verimliliğini yerleştirir.
`+`	Yeni bir ggplot2 katmanı ekler.
`geom_point()`	Her gözlemi nokta olarak çizer.

Neden `x = "disp"` yazmıyoruz? Çünkü `aes()` içinde `disp`, veri setindeki değişkenin adıdır; `"disp"` ise sabit bir metindir.

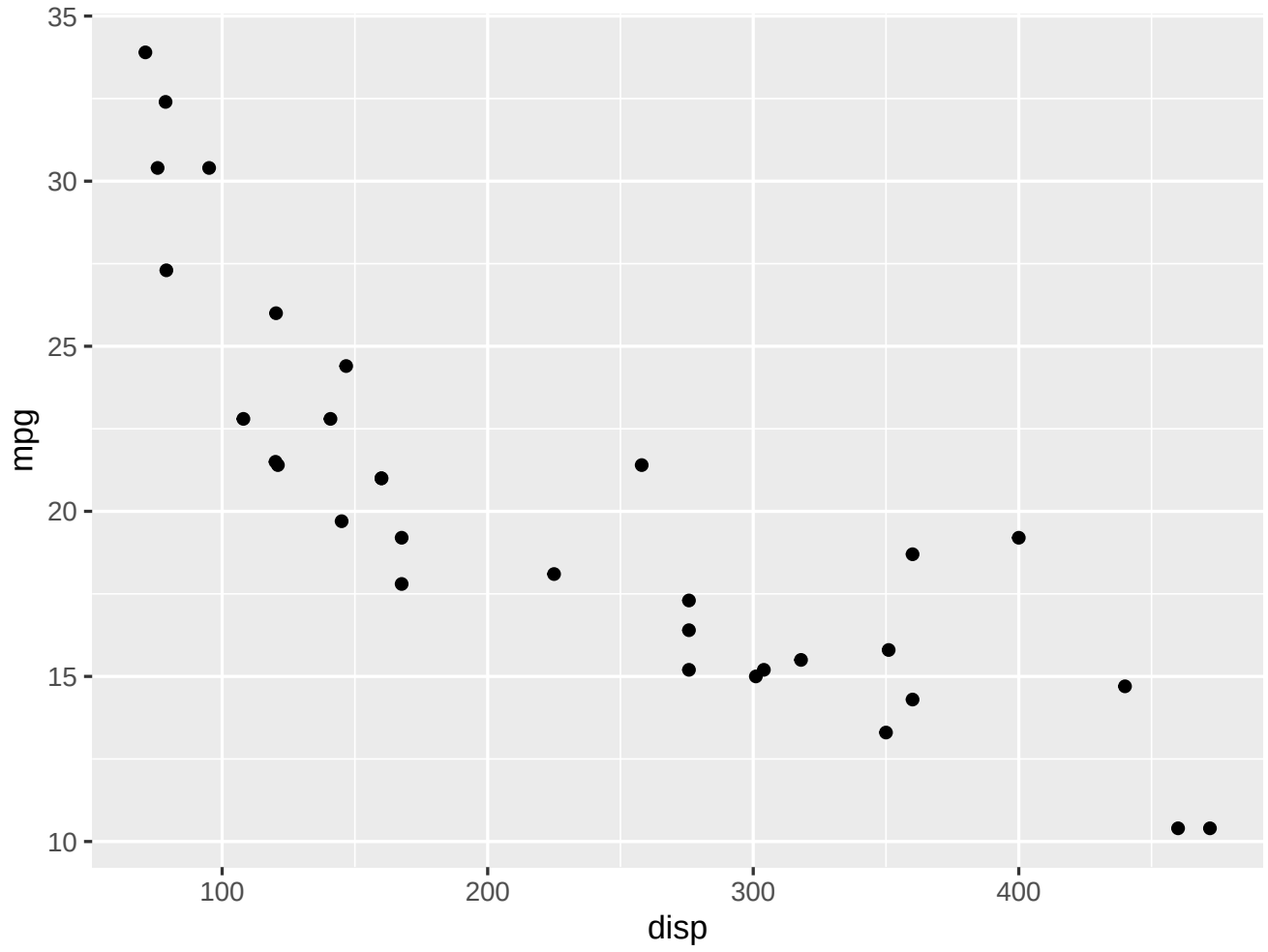
3.1 2.1. Kodun adım adım kurulması

```
# Yalnızca veri tanımlanır; henüz çizilecek geometri yoktur.
ggplot(data = df)
```

```
# Eksenlere değişkenler eşlenir; hâlâ nokta/çizgi katmanı yoktur.
ggplot(data = df, aes(x = disp, y = mpg))
```




```
# Nokta katmanı eklendiğinde gözlemler görünür.
ggplot(data = df, aes(x = disp, y = mpg)) +
  geom_point()
```



Kendin dene: `disp` yerine `wt` yaz. Grafikte hangi ilişkiyi incelemeye başladın?

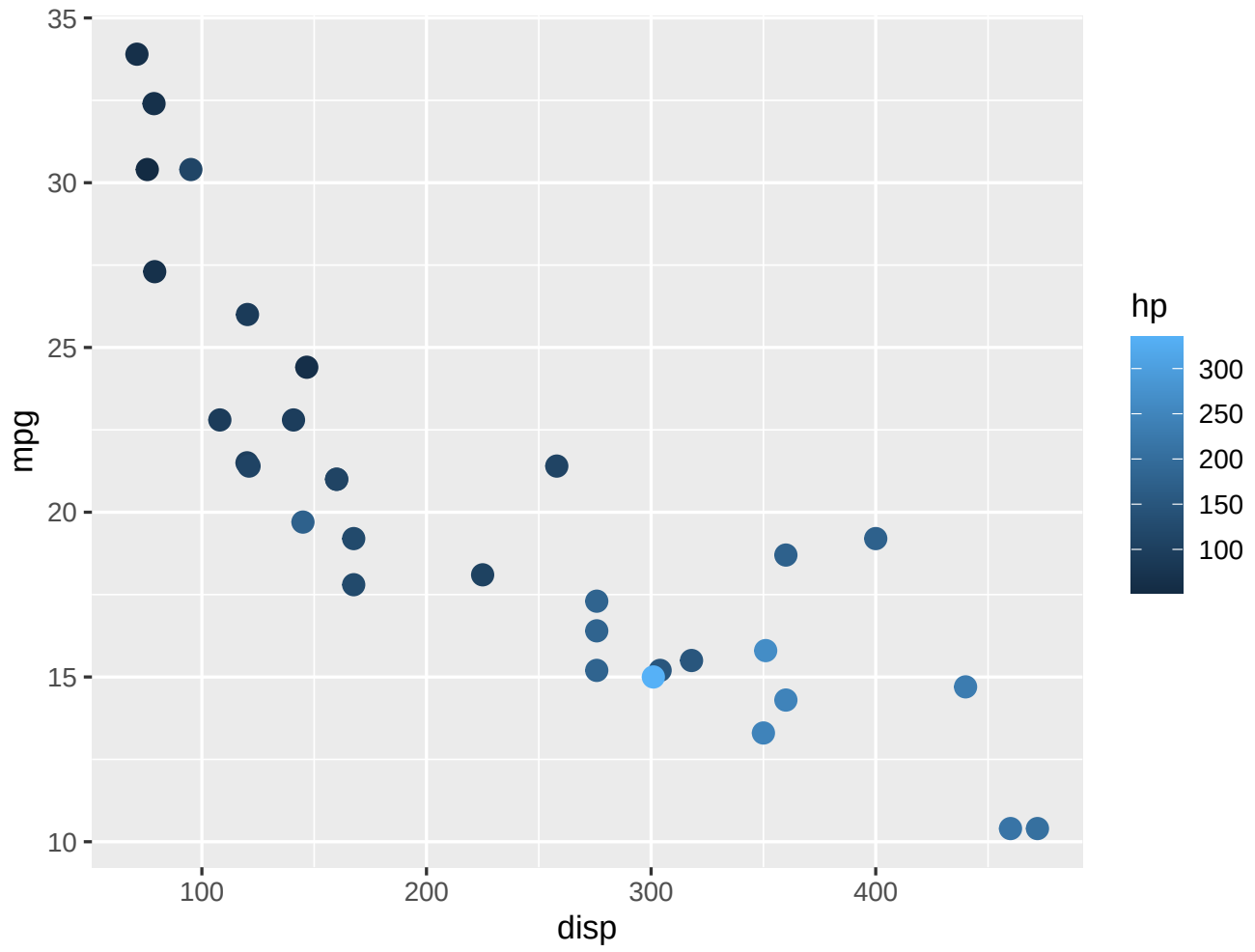
4 3. Aesthetics: Görsel Özellikler

`aes()` içindeki *aesthetic*, "güzel görünüm" değil, veriyi temsil eden görsel kanaldır.

- `x`, `y`: konum
- `color`: nokta/çizgi rengi
- `fill`: sütun, kutu, violin gibi şekillerin iç dolgu rengi
- `size`: büyüklük
- `shape`: nokta şekli
- `alpha`: saydamlık
- `linetype`: çizgi tipi

4.1 3.1. Mapping: Özelliği bir değişkene bağlama

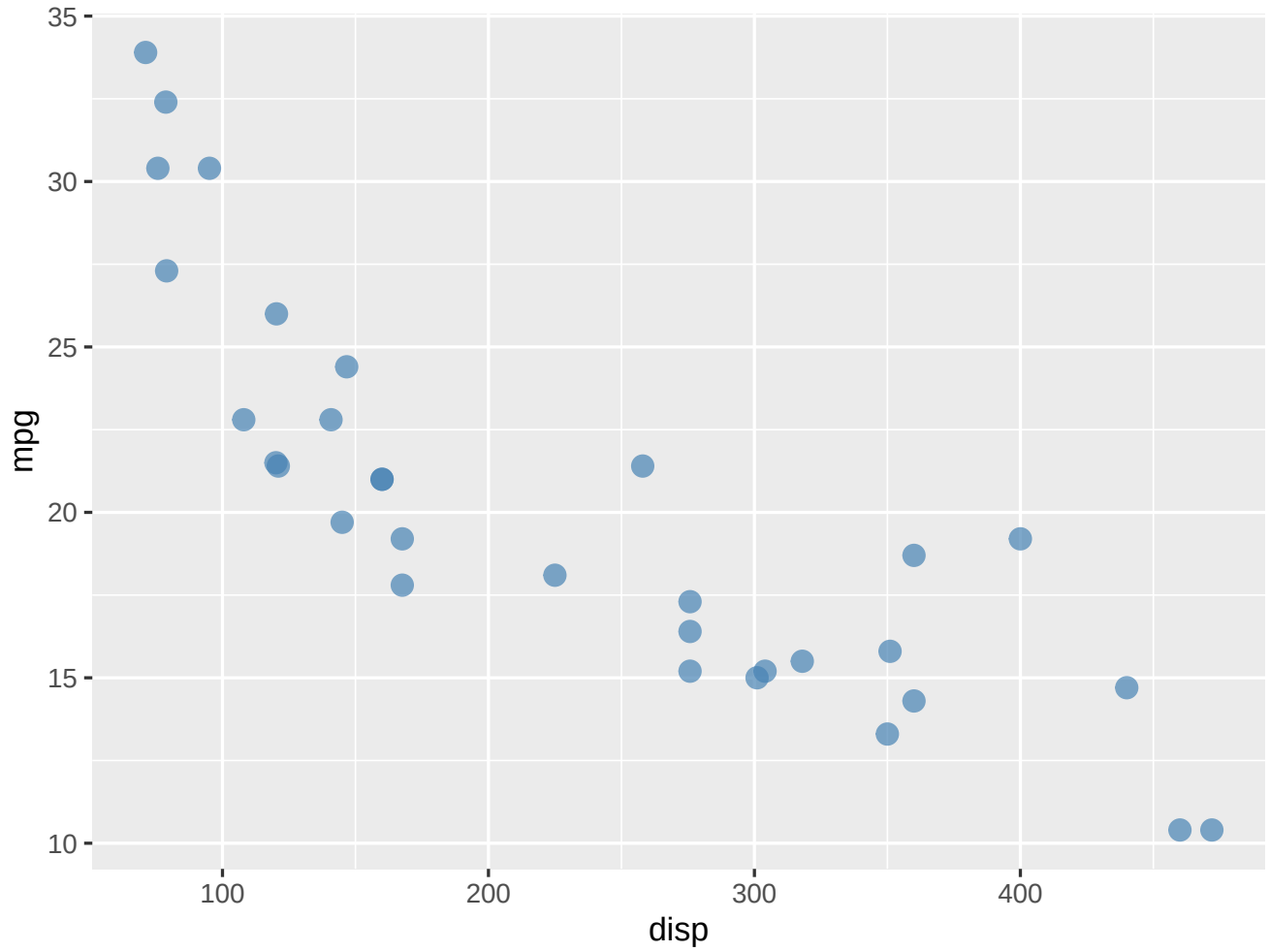
```
ggplot(df, aes(x = disp, y = mpg, color = hp)) +
  geom_point(size = 3)
```



Burada her noktanın rengi `hp` değerine bağlıdır. `hp` sayısal olduğu için varsayılan olarak sürekli bir renk ölçeği oluşur.

4.2 3.2. Setting: Sabit bir görünüm verme

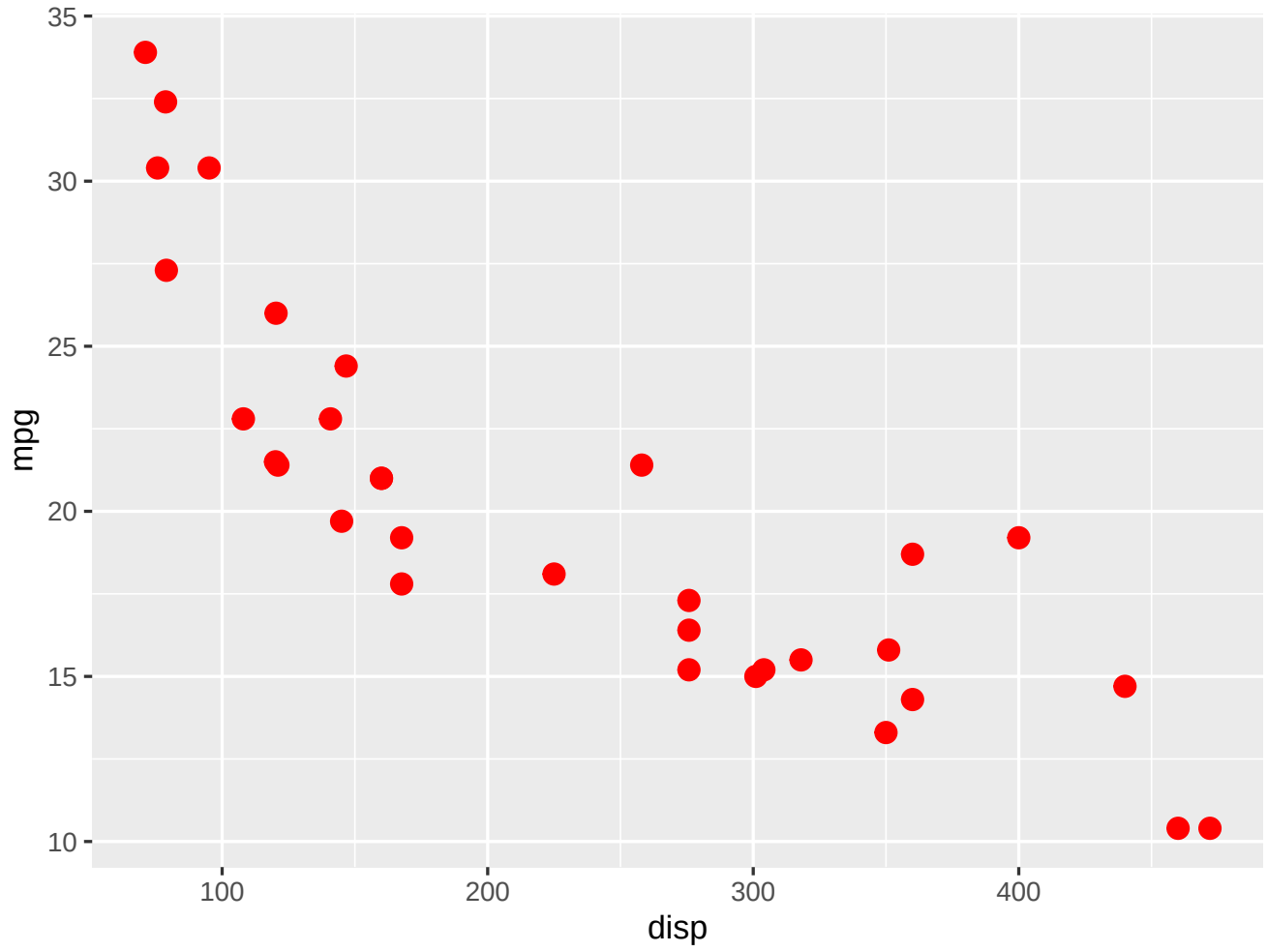
```
ggplot(df, aes(x = disp, y = mpg)) +
  geom_point(color = "steelblue", size = 3, alpha = 0.7)
```



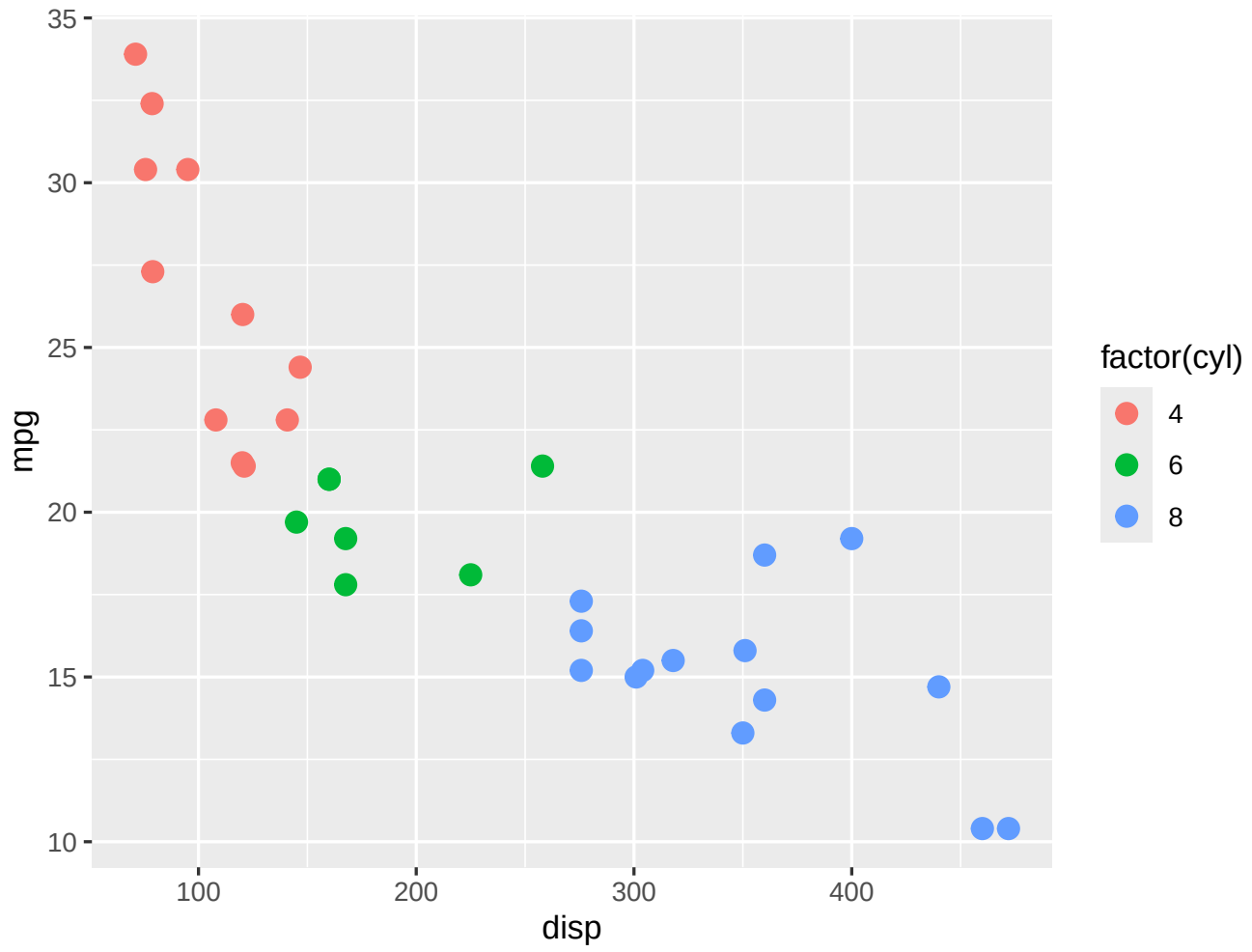
Bütün noktalar aynı renkte ve büyüklüktedir. Çünkü `color`, `size` ve `alpha` bir veri değişkenine bağlanmamıştır.

4.3 3.3. Karşılaştırma: `aes()` içinde ve dışında

```
# Sabit renk: bütün noktalar kırmızı
ggplot(df, aes(disp, mpg)) +
  geom_point(color = "red", size = 3)
```



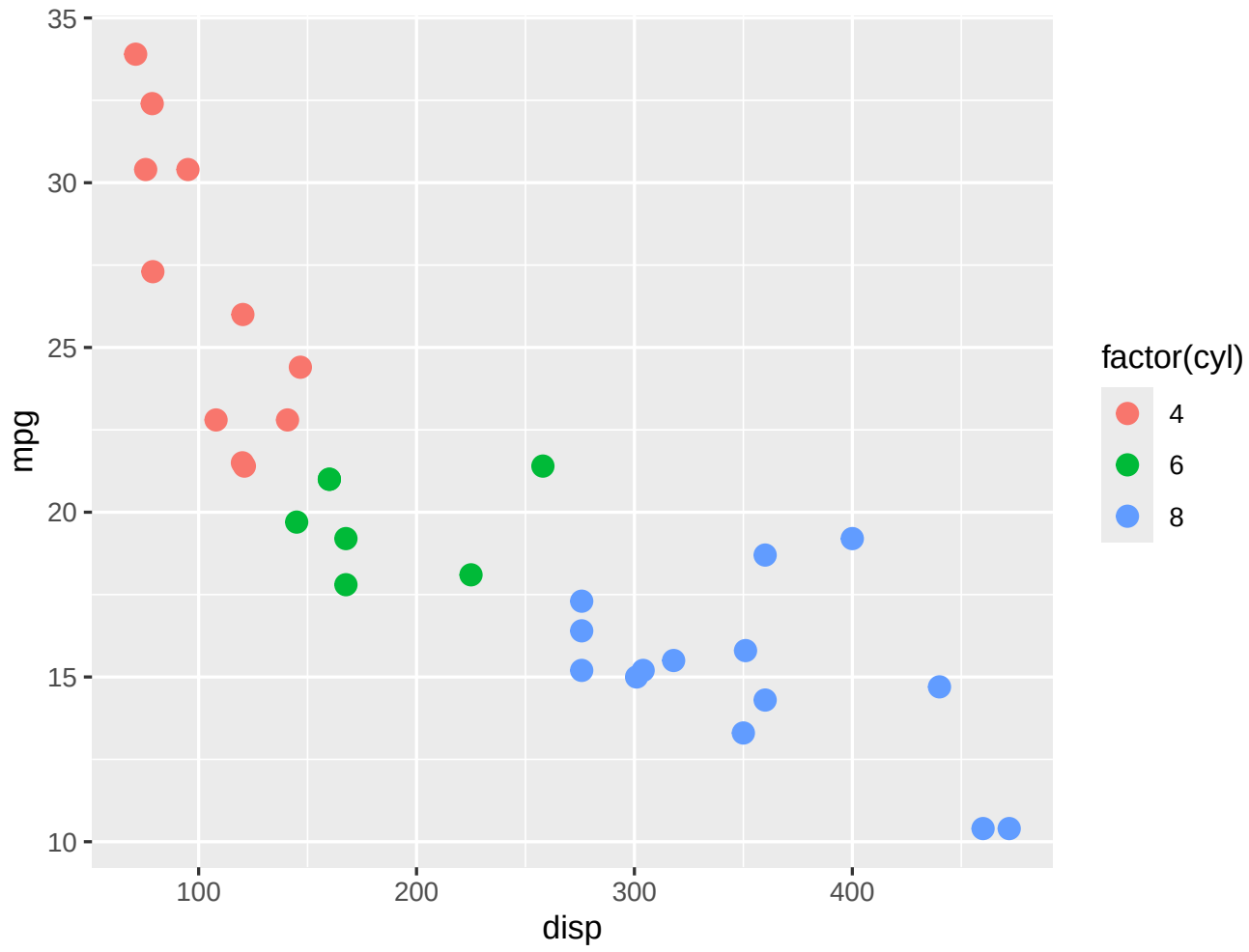

```
# Veriye bağlı renk: silindir grupları farklı renkte
ggplot(df, aes(disp, mpg)) +
  geom_point(aes(color = factor(cyl)), size = 3)
```



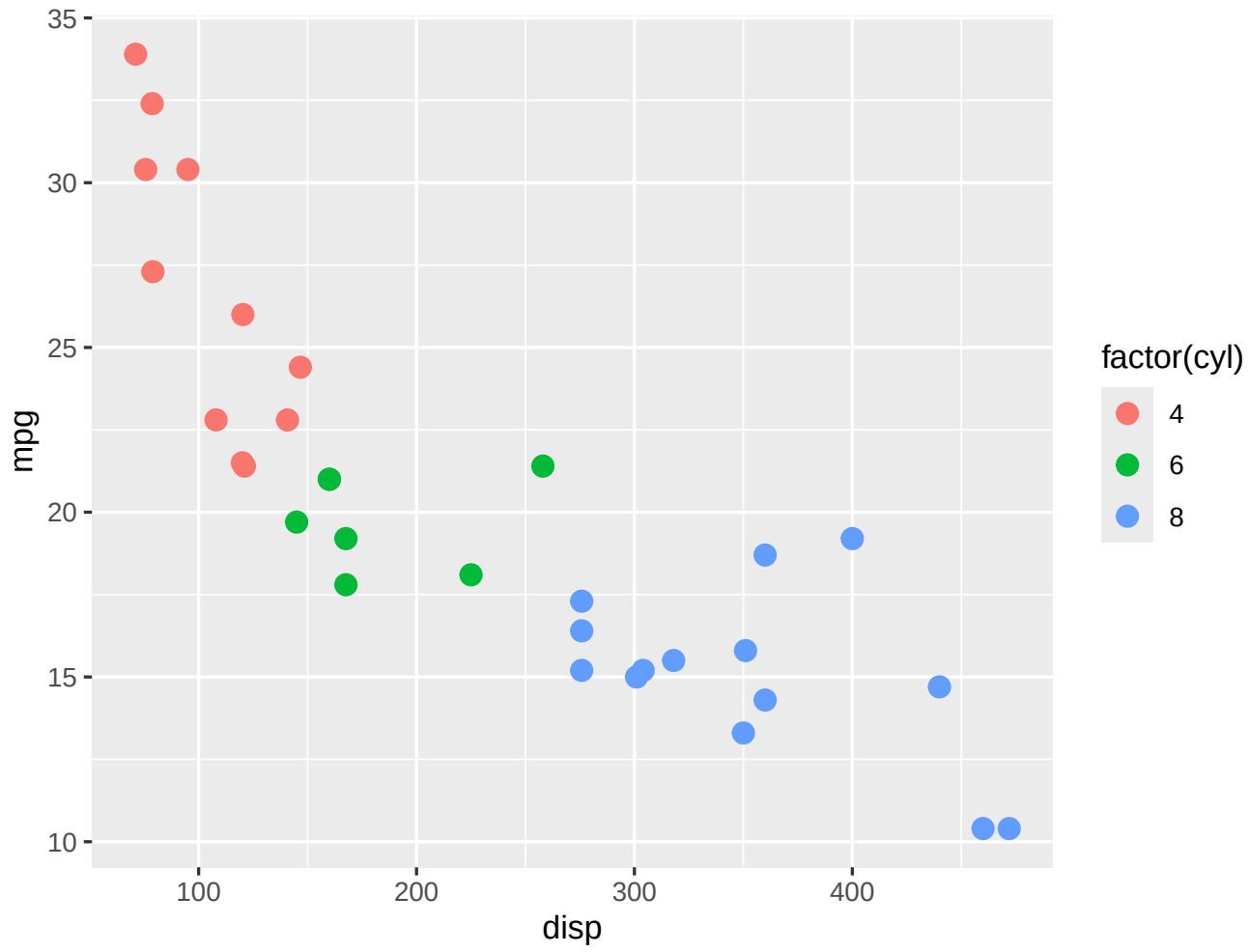
Not: `aes(color = "red")` yazarsanız `"red"` sabit bir kategori etiketi gibi eşlenir; tüm noktaların doğrudan kırmızı olmasını istediğinizde `color = "red"` ifadesini `aes()` dışında kullanın.

4.4 3.4. Global ve yerel eşleştirme

```
# Global: aes() tüm uygun katmanlara aktarılır
ggplot(df, aes(x = disp, y = mpg, color = factor(cyl))) +
  geom_point(size = 3)
```



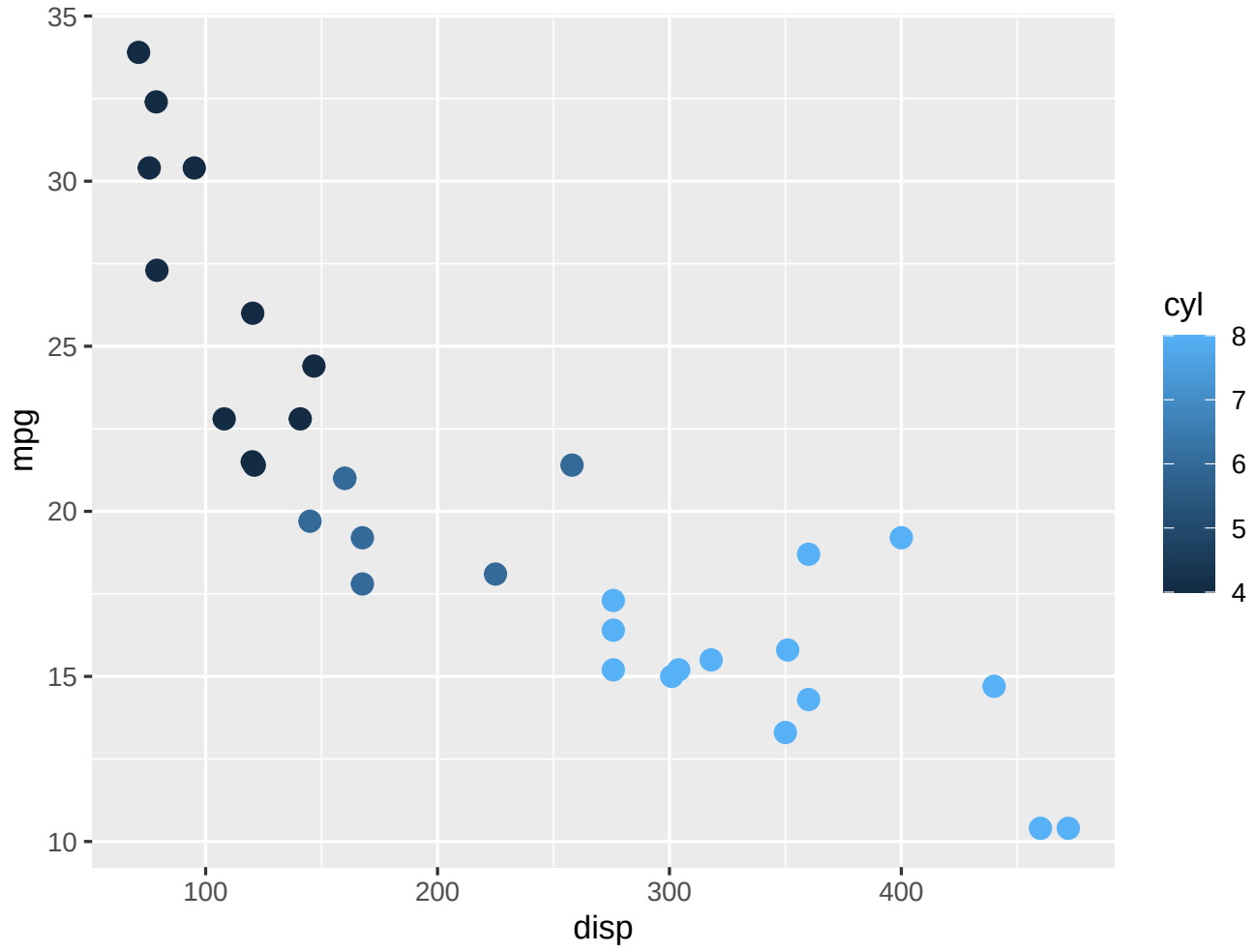

```
# Yerel: renk eşleştirmesi yalnızca bu geom'a aittir
ggplot(df, aes(x = disp, y = mpg)) +
  geom_point(aes(color = factor(cyl)), size = 3)
```



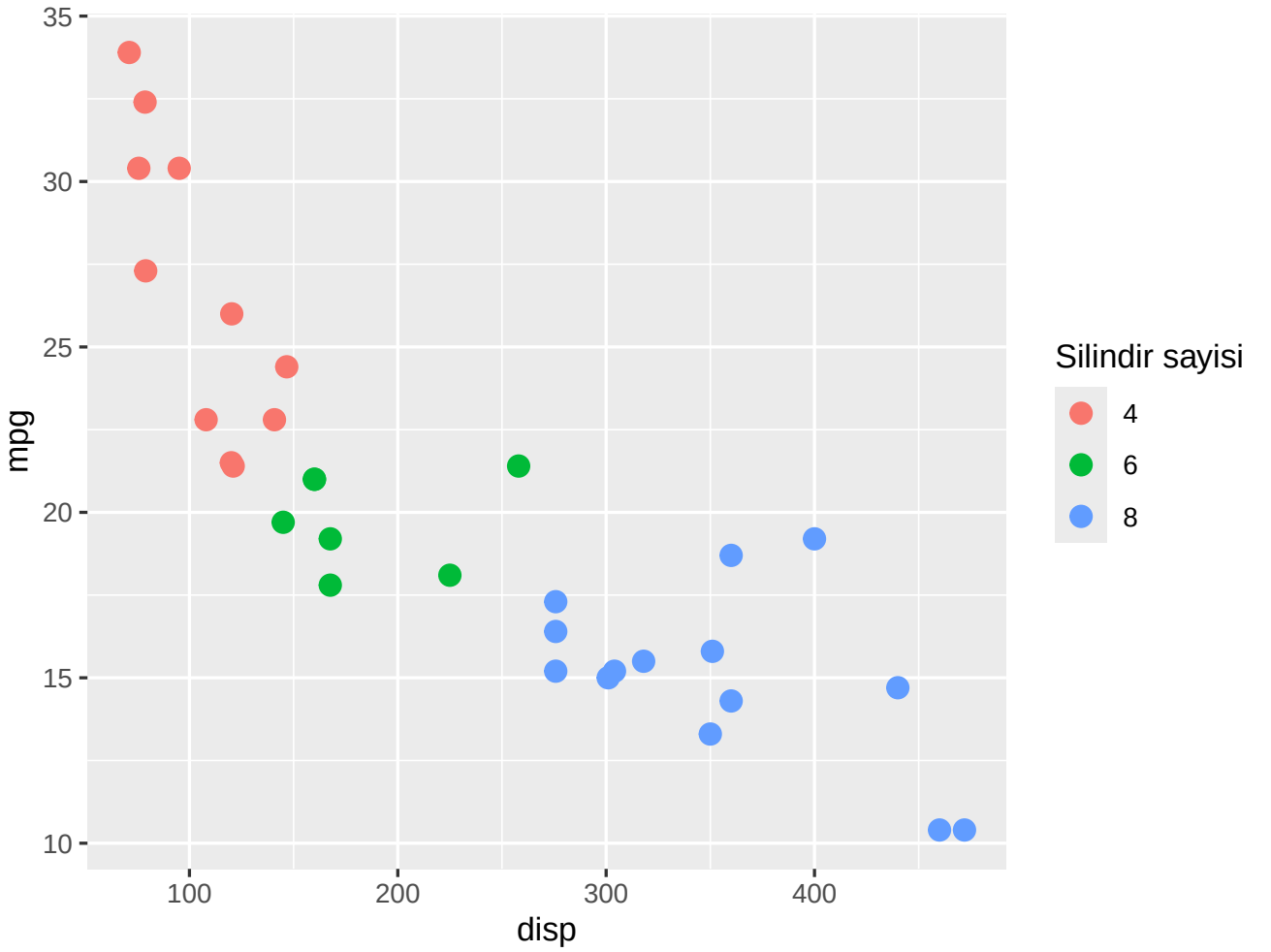
Tek geometri olduğunda sonuç benzerdir, birden fazla katman eklediğimizde fark önem kazanır.

5 4. Kategorik Değişken ve `factor()`

```
# cyl sayısal: sürekli renk ölçeği
ggplot(df, aes(x = disp, y = mpg, color = cyl)) +
  geom_point(size = 3)
```



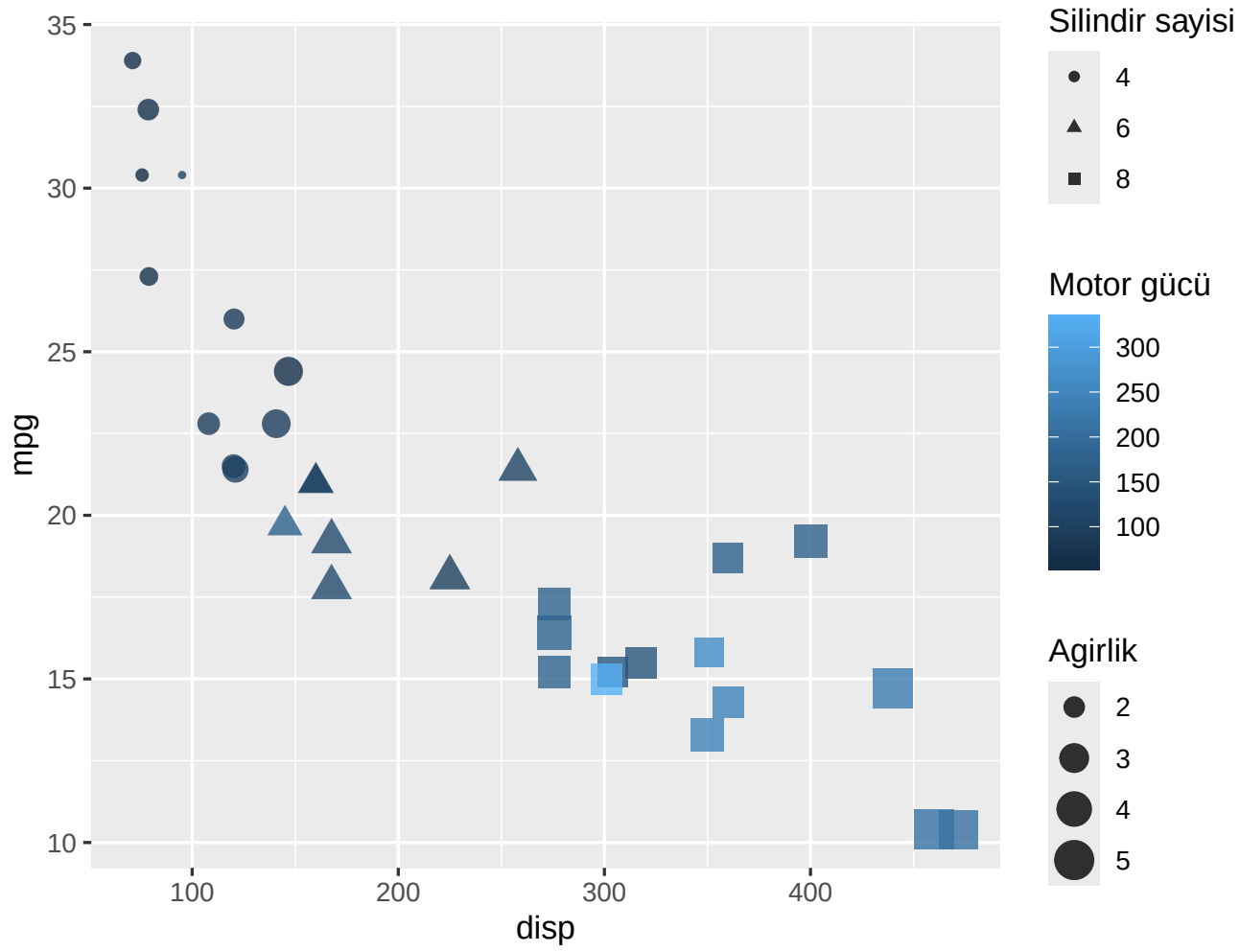

```
# cyl kategorik: 4, 6 ve 8 ayrı sınıflar
ggplot(df, aes(x = disp, y = mpg, color = factor(cyl))) +
  geom_point(size = 3) +
  labs(color = "Silindir sayısı")
```



Yorum: Bir değişkenin R ortamında saklandığı veri türü, analizde nasıl ele alınması gerektiğini her zaman belirlemez. Bu nedenle, değişkenlerin özelliklerine ve görselleştirme amacına bağlı olarak veri türlerinin dönüştürülmesi gerekebilir. Örneğin, mtcars veri setindeki cyl değişkeni sayısal (numeric) olarak saklanmasına rağmen 4, 6 ve 8 silindirli araç gruplarını temsil etmektedir. Bu değişkeni kategorik olarak görselleştirmek istediğimizde factor() fonksiyonunu kullanabilir

6 5. color, fill, shape, size, alpha

```
# Renk motor gücünü, boyut ağırlığı, şekil silindir grubunu temsil ediyor.
ggplot(df, aes(x = disp, y = mpg,
               color = hp, size = wt, shape = factor(cyl))) +
  geom_point(alpha = 0.8) +
  labs(color = "Motor gücü", size = "Ağırlık",
       shape = "Silindir sayısı")
```



Çok sayıda görsel kanal eklemek daha fazla bilgi taşıyabilir, ancak okunabilirliği azaltabilir.

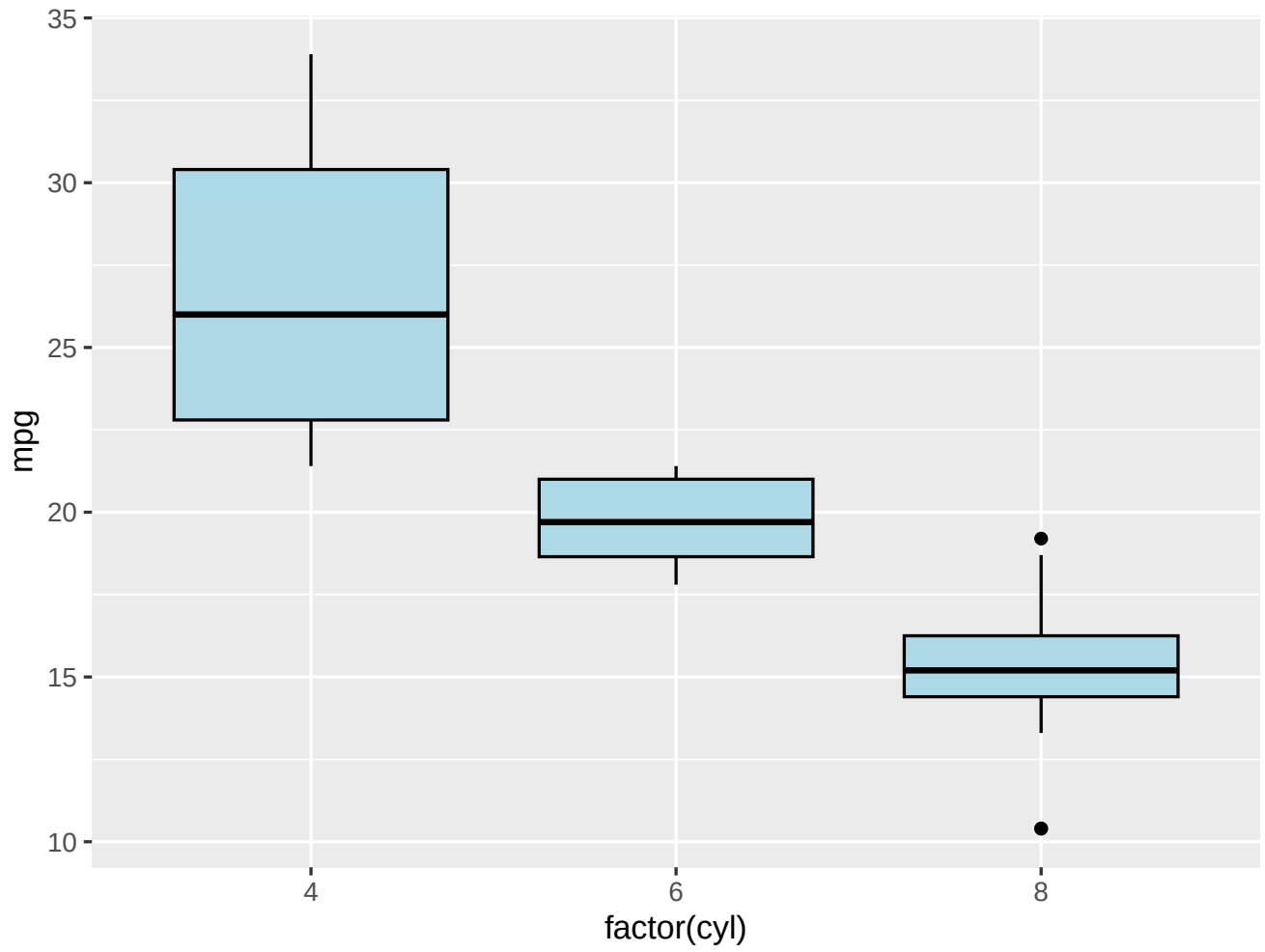
6.0.1 `fill` ve `color` Arasındaki Fark

`ggplot2` paketinde `fill` ve `color`, grafik öğelerinin renklerini belirlemek için kullanılan iki farklı görsel özelliktir.

- **`fill`:** Grafik öğelerinin iç kısmının rengini belirler.
- **`color`:** Grafik öğelerinin kenar veya çizgi rengini belirler.

Örneğin, bir kutu grafiğinde (`geom_boxplot()`) kutuların içini açık mavi, kenar çizgilerini ise siyah renkte gösterebiliriz.

```
ggplot(mtcars, aes(x = factor(cyl), y = mpg)) +
  geom_boxplot(fill = "lightblue", color = "black")
```

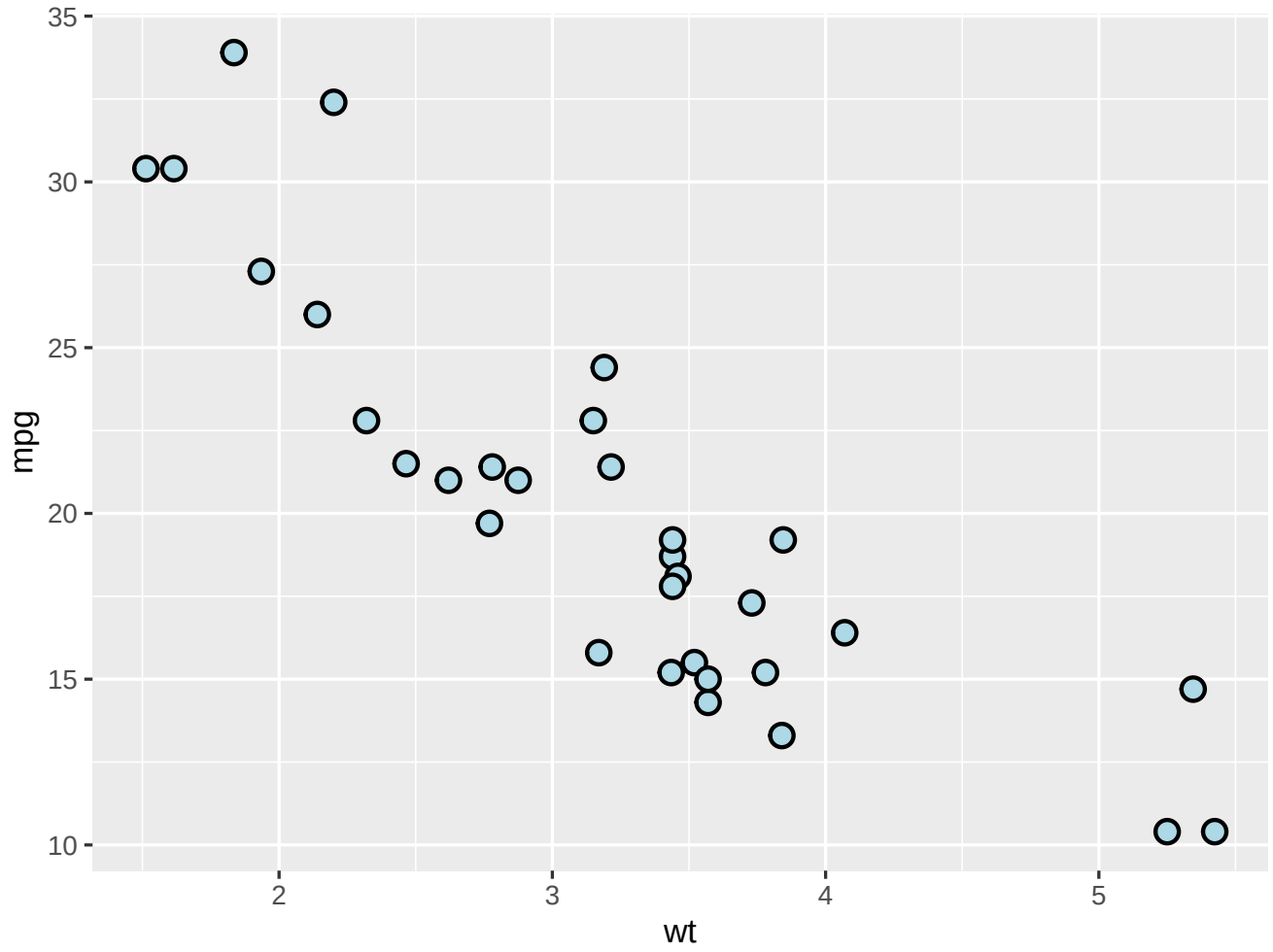


Kodun açıklaması:

- `fill = "lightblue"`: Kutuların iç kısmını açık mavi renkte gösterir.
- `color = "black"`: Kutuların kenar çizgilerini siyah renkte gösterir.

Önemli Not: Nokta grafiklerinde (`geom_point()`) `color` genellikle noktaların rengini belirler. Ancak `shape = 21` gibi içi doldurulabilen nokta şekillerinde `fill` noktanın iç rengini, `color` ise kenar rengini kontrol eder.

```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point(
    shape = 21,
    fill = "lightblue",
    color = "black",
    size = 3,
    stroke = 1
  )
```



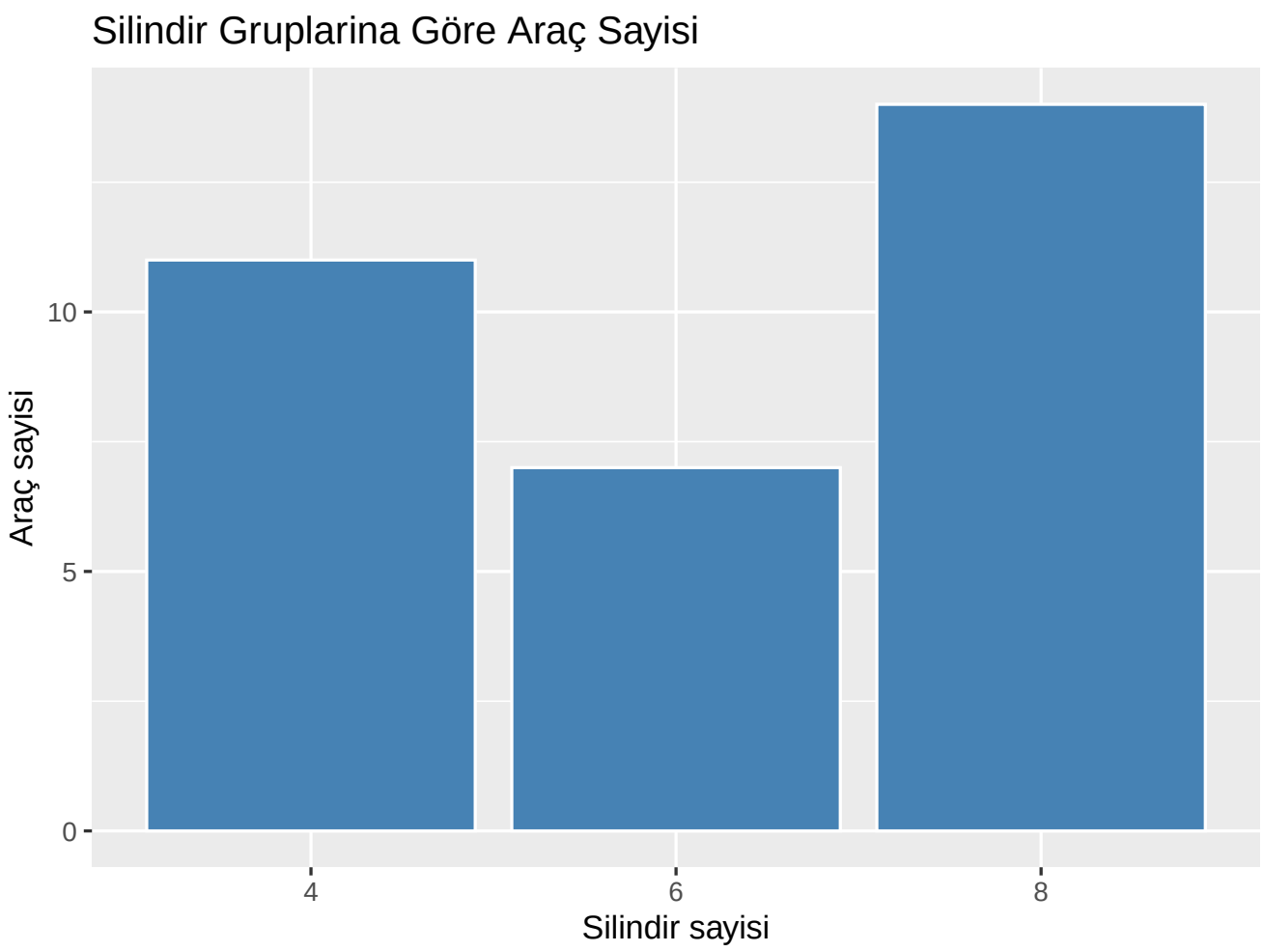
Bu örnekte noktaların içi açık mavi, kenarları ise siyah renktedir. `stroke` argümanı, noktaların kenar çizgilerinin kalınlığını belirler..

7 6. `geom_bar()`: Sütun Grafiği

Ne zaman kullanılır? Kategorilerin kaç gözlem içerdiğini göstermek için.

`geom_bar()` varsayılan olarak `stat = "count"` kullanır; her kategori için gözlem sayısını **kendisi hesaplar**. Bu nedenle sadece `x` tanımlamak yeterlidir.

```
ggplot(df, aes(x = factor(cyl))) +
  geom_bar(fill = "steelblue", color = "white") +
  labs(x = "Silindir sayısı", y = "Araç sayısı",
       title = "Silindir Gruplarına Göre Araç Sayısı")
```

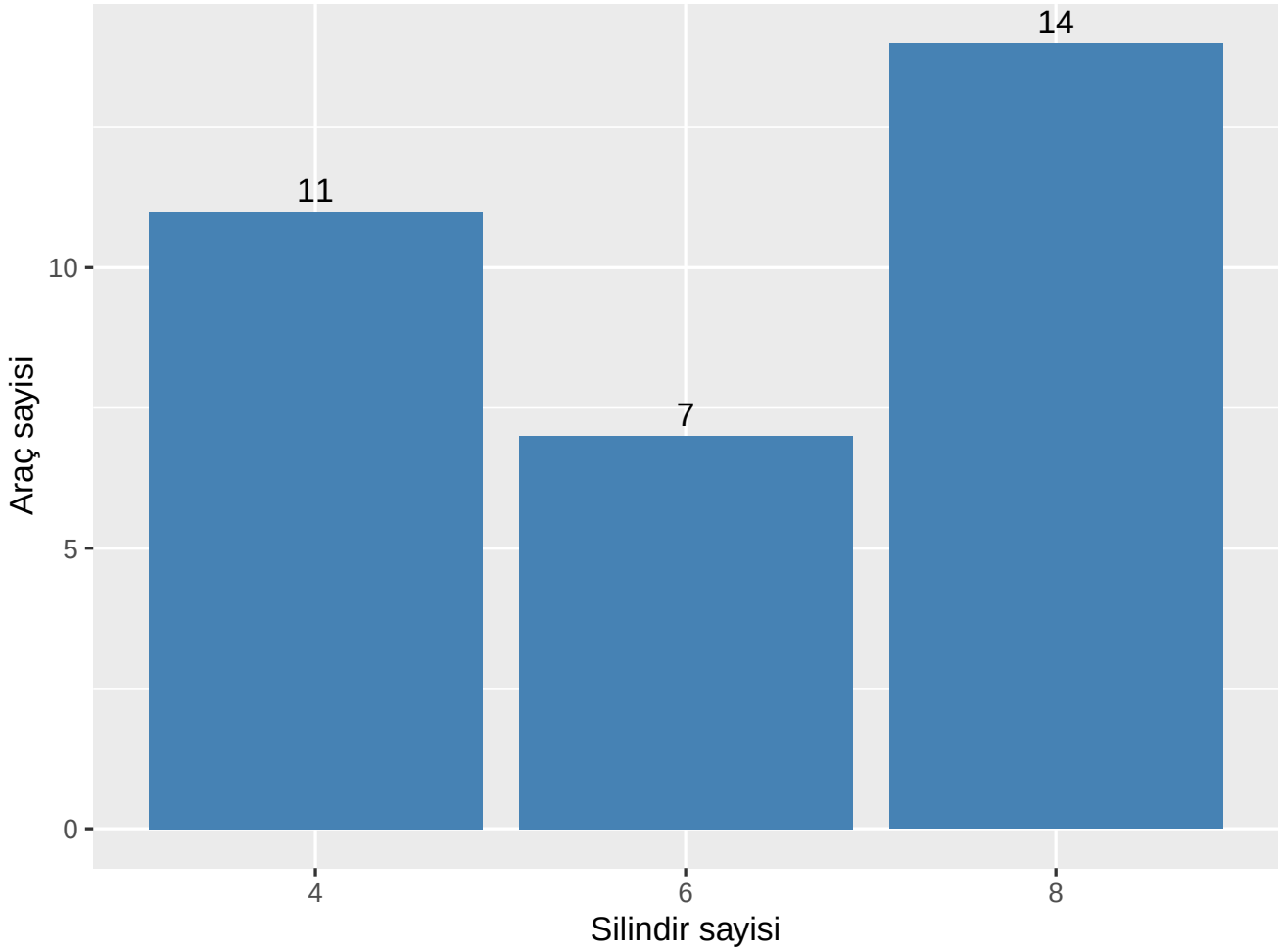



```
table(df$cyl)
#>
#>  4  6  8
#> 11  7 14
```

Sonuç: 4 silindirli **11**, 6 silindirli **7**, 8 silindirli **14** otomobil.

7.1 6.1. Sütunların üzerine sayıları yazmak: `geom_text()`

```
ggplot(df, aes(x = factor(cyl))) +
  geom_bar(fill = "steelblue") +
  geom_text(stat = "count",
            aes(label = after_stat(count)),
            vjust = -0.4) +
  labs(x = "Silindir sayısı", y = "Araç sayısı")
```



- `stat = "count"`: Etiket katmanı için kategori sayısını hesaplar.
- `after_stat(count)`: Hesaplanan sayıyı etiket olarak kullanır.
- `vjust = -0.4`: Etiketi sütunun üstüne taşır.
- Eski `..count..` gösterimi yerine `after_stat(count)` tercih edilir.

7.1.1 `geom_bar()` ve `geom_col()` Arasındaki Fark

`geom_bar()` ve `geom_col()` fonksiyonları sütun grafiği oluşturmak için kullanılır. Ancak aralarındaki temel fark, **sütun yüksekliklerinin nasıl belirlendiğidir.**

- **`geom_bar()`**: "Bana kategorileri söyle, her kategoride kaç gözlem olduğunu ben sayarım."
- **`geom_col()`**: "Benim zaten `x` ve `y` değerlerim var. Yeniden hesaplama yapma, `y` sütunundaki değerleri doğrudan kullan."

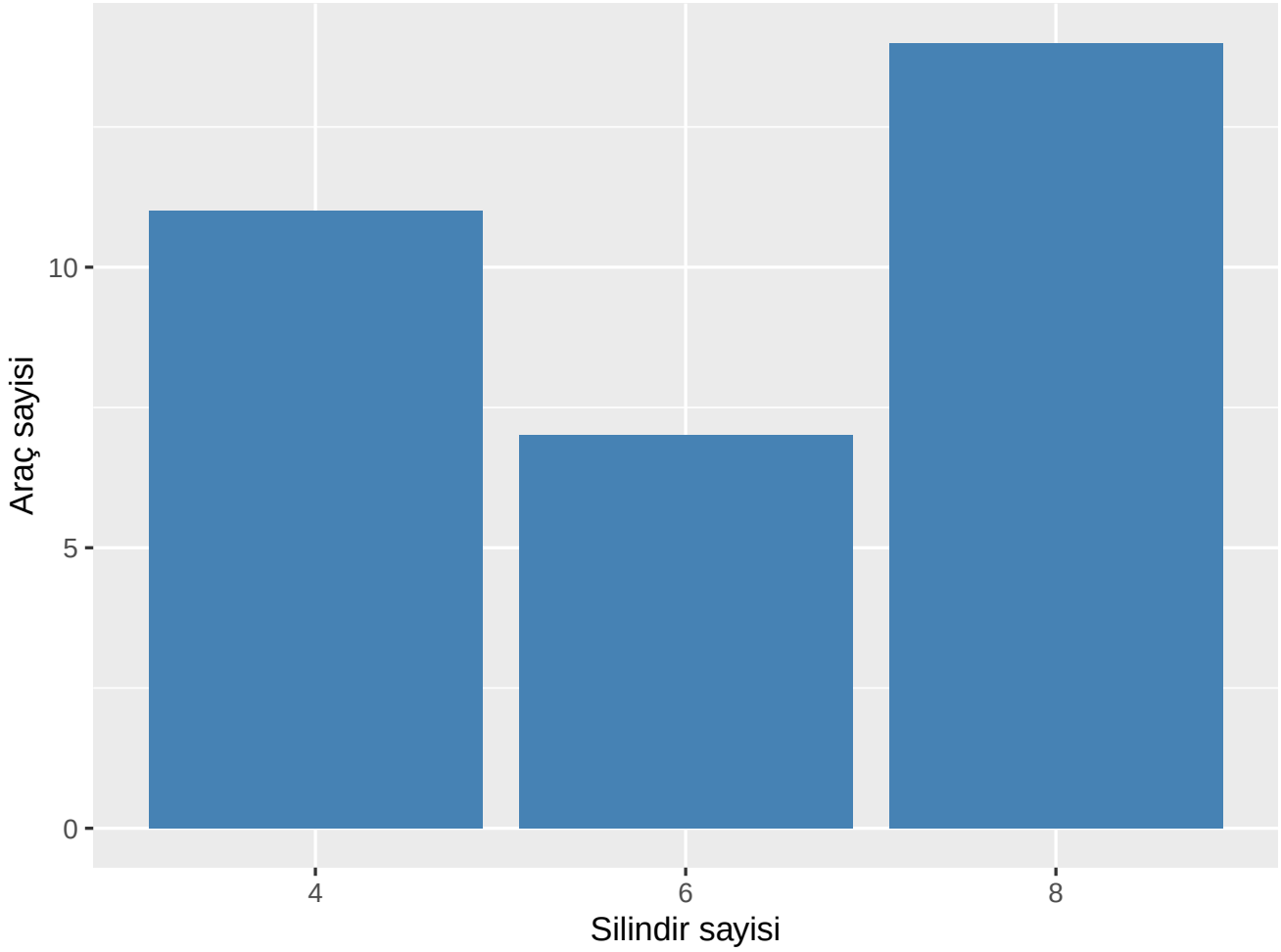
Kısacası:

- `geom_bar()` gözlem sayılarını kendisi hesaplar.
- `geom_col()` veri setinde bulunan `y` değerlerini doğrudan kullanır.

```r
# Her silindir grubundaki araç sayısını kendisi hesaplar

ggplot(mtcars, aes(x = factor(cyl))) +
  geom_bar(fill = "steelblue") +
  labs(
    x = "Silindir sayısı",
    y = "Araç sayısı"
  )
```

7.1.1.1 Örnek 1: `geom_bar()`



Bu örnekte yalnızca `x` değişkenini belirttik. `geom_bar()` her silindir grubunda kaç otomobil bulunduğunu kendisi hesapladı.

7.1.1.2 Örnek 2: `geom_col()`

Önceden hesaplanmış araç sayılarını içeren bir veri setimiz olduğunu düşünelim.

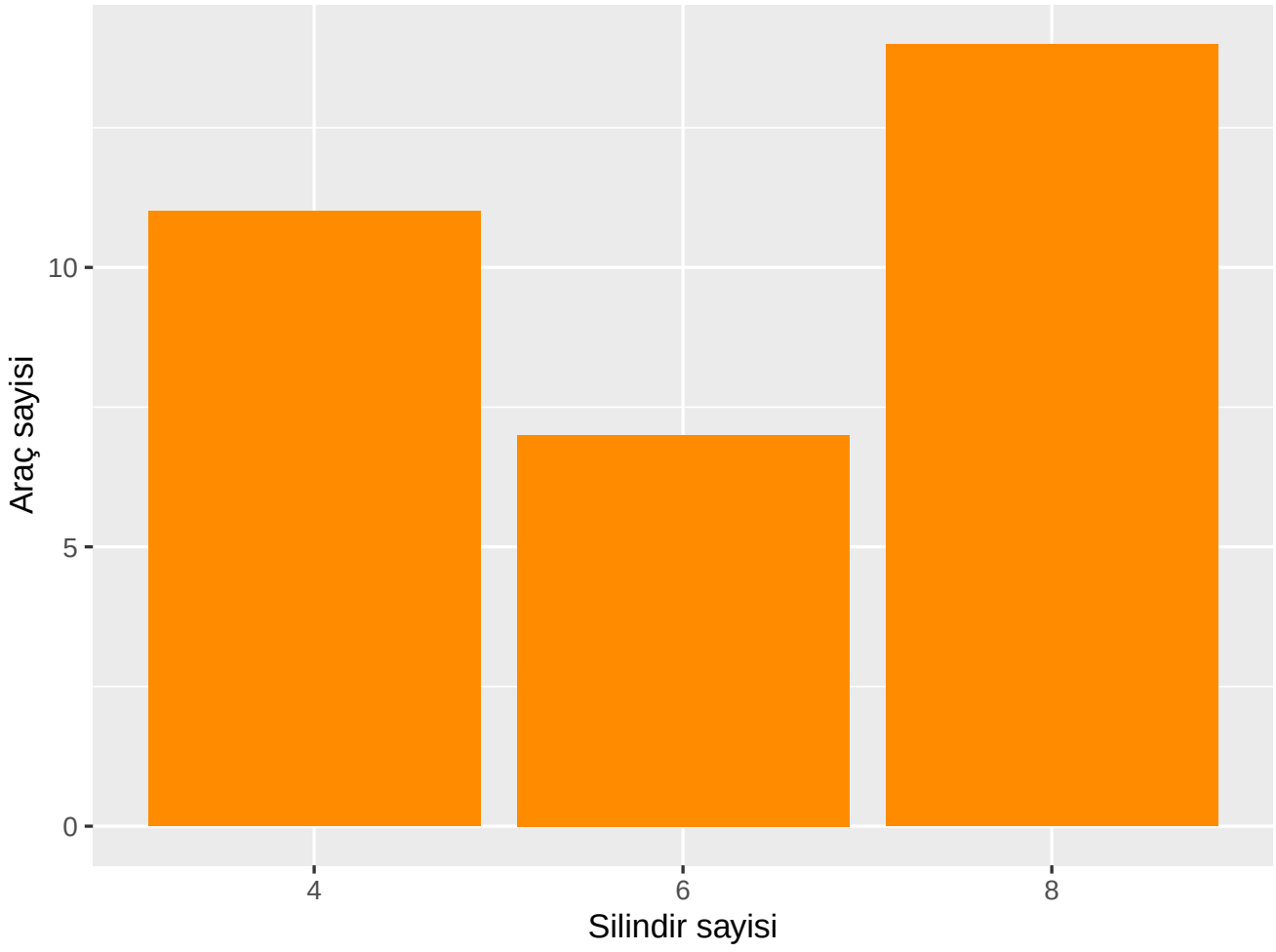
```r
# Araç sayılarını içeren özet tablo

ozet <- data.frame(
  silindir = c("4", "6", "8"),
  arac_sayisi = c(11, 7, 14)
)
```

```
# Veri setini görüntüleme
ozet
#>   silindir arac_sayisi
#> 1        4          11
#> 2        6           7
#> 3        8          14
```

Şimdi bu değerleri `geom_col()` ile görselleştirelim.

```
ggplot(ozet, aes(x = silindir, y = arac_sayisi)) +
  geom_col(fill = "darkorange") +
  labs(
    x = "Silindir sayısı",
    y = "Araç sayısı"
  )
```



Kodun açıklaması:

- `x = silindir`: Yatay eksende silindir gruplarını gösterir.
- `y = arac_sayisi`: Sütunların yüksekliğini belirler.
- `geom_col()`: Araç sayılarını yeniden hesaplamadan doğrudan kullanır.

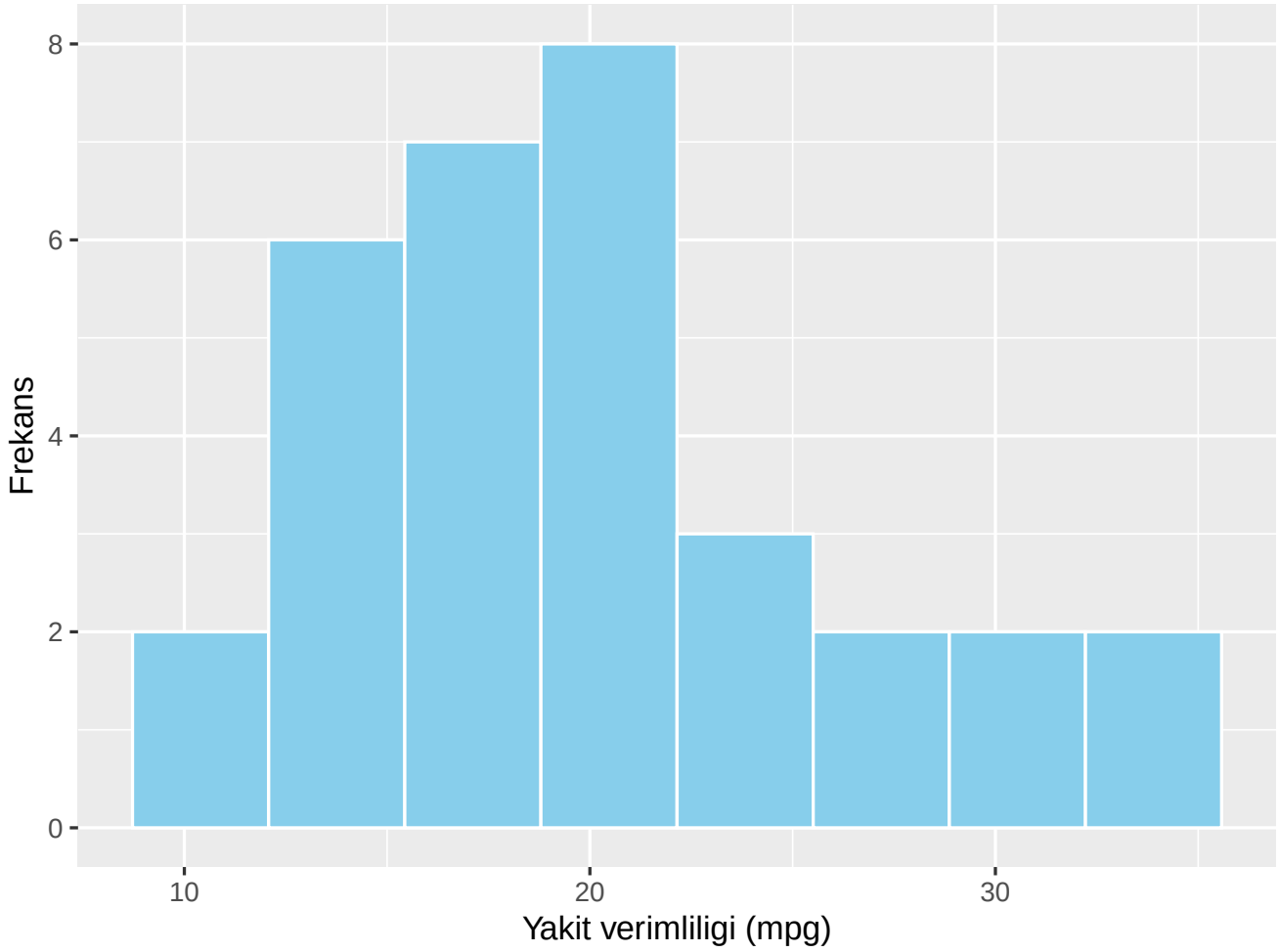
Önemli: Her iki grafik de aynı araç sayılarını göstermektedir. Ancak ilk grafikte sayıları `geom_bar()` hesaplamış, ikinci grafikte ise önceden hesaplanmış değerler `geom_col()` tarafından kullanılmıştır.

8 7. geom_histogram(): Histogram

Ne zaman kullanılır? Sayısal bir değişkenin dağılımını aralıklara bölerek incelemek için.

Histogramdaki sütunlar **kategori** değil, sayısal değer **aralıklarıdır**.

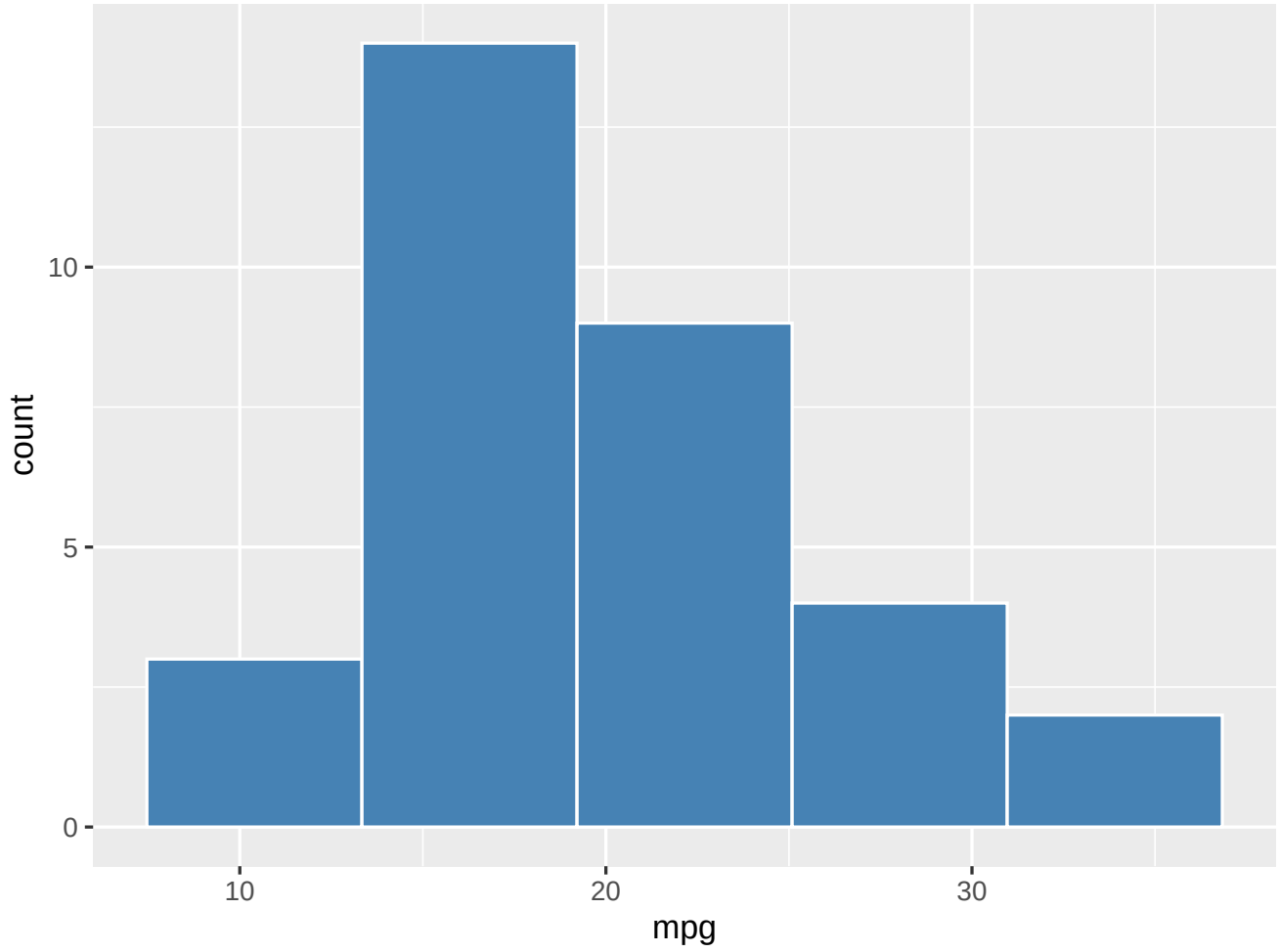
```
ggplot(df, aes(x = mpg)) +
  geom_histogram(bins = 8, fill = "skyblue",
                 color = "white") +
  labs(x = "Yakıt verimliliği (mpg)", y = "Frekans")
```



8.1 7.1. bins ne demek?

`bins = 8`, verinin yaklaşık sekiz aralığa (sınıf) bölünmesini ister. **Her aralıkta sekiz gözlem bulunacağı anlamına gelmez.**

```
ggplot(df, aes(x = mpg)) +
  geom_histogram(bins = 5, fill = "steelblue",
                 color = "white")
```



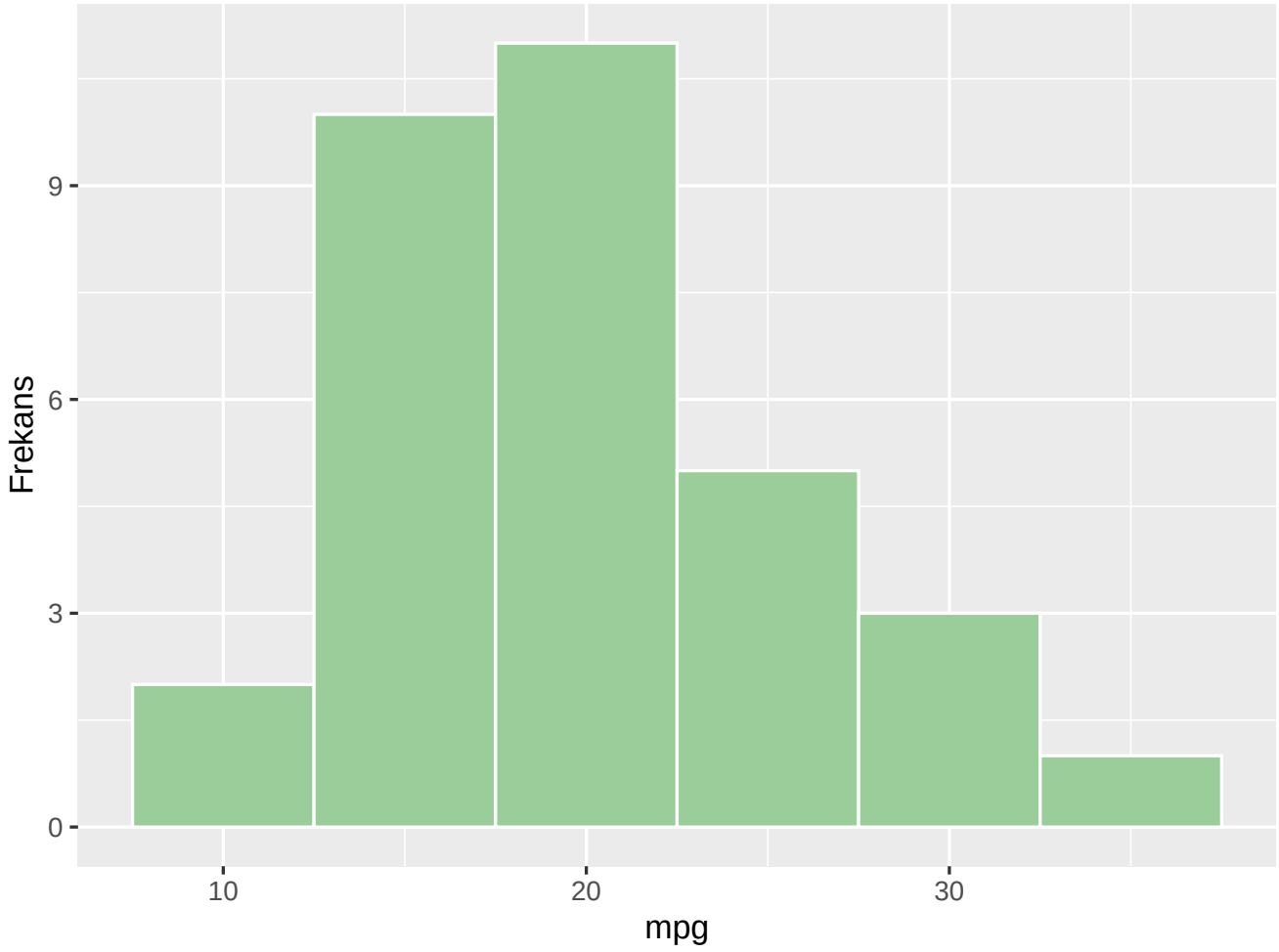
8.2 7.2. `binwidth` nedir?

`binwidth`, histogramda kullanılan aralıkların genişliğini belirler.

Örneğin `binwidth = 5` yazdığımızda, R'den `mpg` değerlerini 5 birim genişliğindeki aralıklara ayırmasını isteriz.

Her sütun bir değer aralığını temsil eder. Sütunun yüksekliği ise o aralıkta kaç gözlem bulunduğunu gösterir.

```
ggplot(df, aes(x = mpg)) +
  geom_histogram(
    binwidth = 5,
    fill = "darkseagreen3",
    color = "white"
  ) +
  labs(x = "mpg", y = "Frekans")
```



Önemli: `binwidth = 5`, her sütunda 5 gözlem bulunduğu anlamına gelmez. Her sütunun yatay eksende 5 birimlik bir aralığı temsil ettiği anlamına gelir.

```
range(df$mpg)
#> [1] 10.4 33.9
```

8.3 7.3. Histogramın yorumlanması

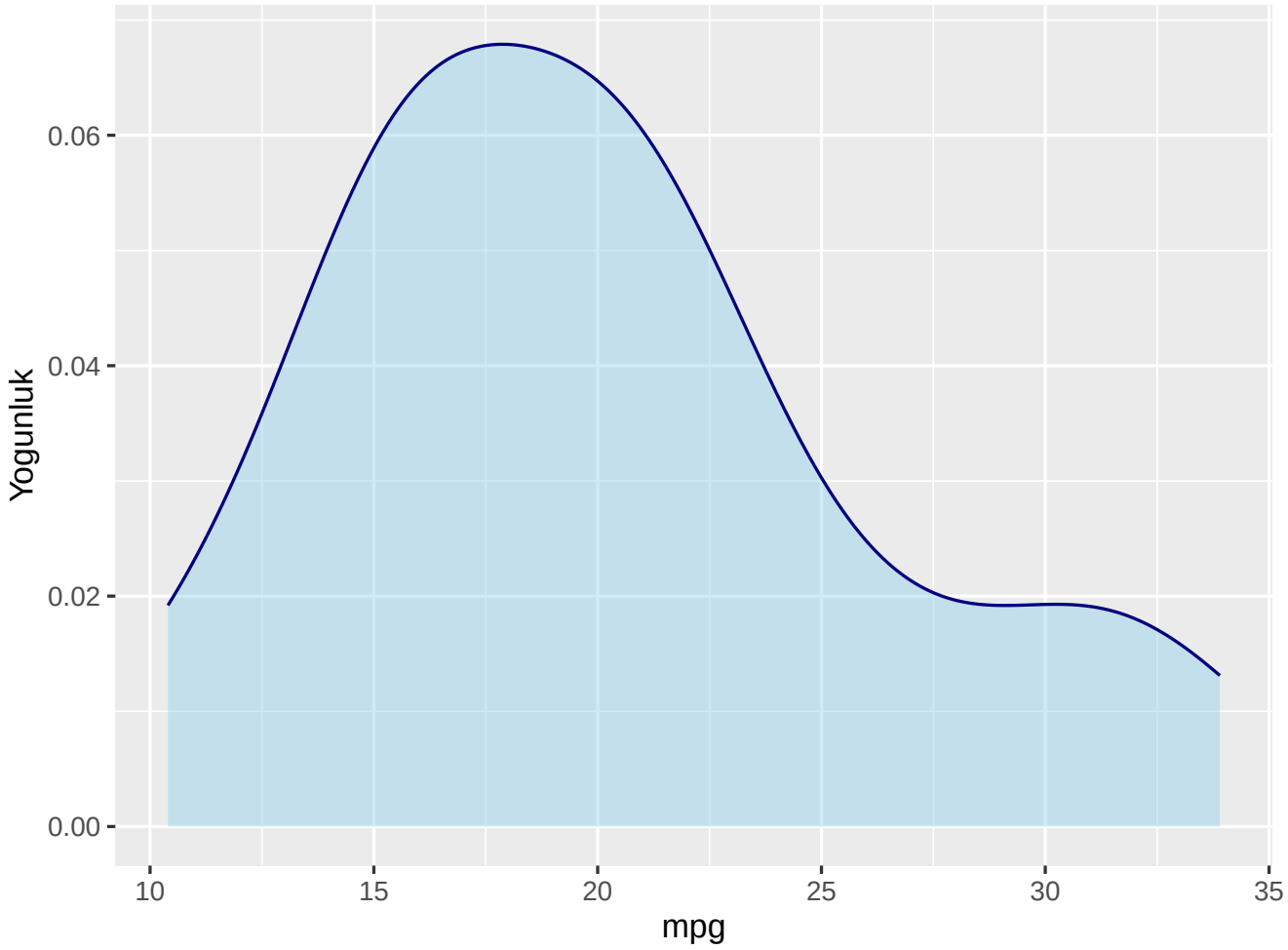
- Hangi aralıkta daha çok gözlem var?
- Dağılım simetrik mi, çarpık mı?
- Tek tepe mi, birden fazla tepe mi görülüyor?
- Aralık genişliği değişince grafik nasıl değişiyor?

Kendiniz deneyin: `bins = 5`, `bins = 10` ve `binwidth = 3` için üç histogram çizip karşılaştırın.

9 8. `geom_density()`: Yoğunluk Eğrisi

Ne zaman kullanılır? Sürekli sayısal bir değişkenin dağılımının daha pürüzsüz bir özetini görmek için.

```r
ggplot(df, aes(x = mpg)) +
  geom_density(fill = "skyblue", alpha = 0.4,
               color = "navy") +
  labs(x = "mpg", y = "Yoğunluk")
```



Yoğunluk eğrisinin dikey ekseni **gözlem sayısı değildir**. Yoğunluk eğrisi altındaki toplam alan 1 olacak şekilde ölçeklenir. Bir aralıktaki eğri altı alan, o aralığın yaklaşık olasılık payını temsil eder.

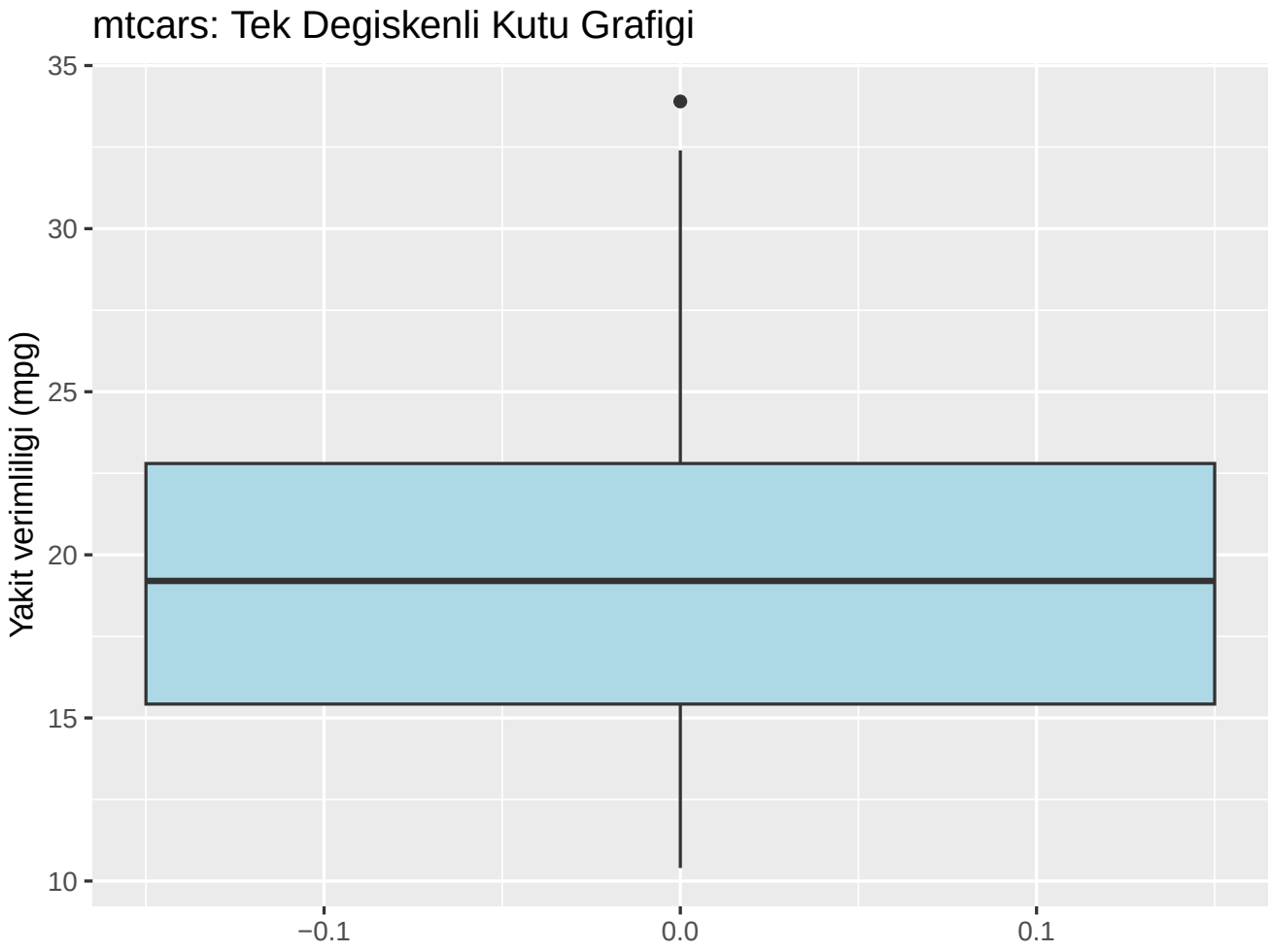
10 9. `geom_boxplot()`: Kutu Grafiği

Ne zaman kullanılır? Sayısal bir değişkenin merkezini, yayılımını ve olası aykırı değerlerini özetlemek ve grupları karşılaştırmak için.

Kutu grafiğinde temel parçalar:

- **Q1:** Birinci çeyrek (%25'lik çeyrek).
- **Medyan (Q2):** Ortanca (%50'lik çeyrek).
- **Q3:** Üçüncü çeyrek (%75'lik çeyrek).
- **IQR:** Çeyrekler arası açıklık, `Q3 - Q1`.
- **Sınırlar:** Genellikle 1.5 × IQR sınırları **içinde kalan en uç gözlemlere** kadar uzanır.
- **Sınırların dışındaki noktalar:** 1.5 × IQR kuralına göre işaretlenen olası aykırı değerlerdir; mutlaka hatalı veri değildir.

```r
ggplot(df, aes(y = mpg)) +
  geom_boxplot(fill = "lightblue", width = 0.3) +
  labs(y = "Yakıt verimliliği (mpg)",
       title = "mtcars: Tek Değişkenli Kutu Grafiği")
```



10.1 9.1. Çeyrekler ve sınırlar nasıl hesaplanır?

Önce küçük bir örnek veri seti düşünelim:

```r
ornek <- c(1, 2, 3, 4, 5, 6, 7, 8, 20)
sort(ornek)
#> [1]  1  2  3  4  5  6  7  8 20
quantile(ornek, probs = c(0.25, 0.5, 0.75))
#> 25% 50% 75%
#>   3   5   7
```

Bu örnek için R'nin varsayılan `quantile()` yöntemiyle:

- `Q1 = 3`
- `Medyan = 5`
- `Q3 = 7`
- `IQR = 7 - 3 = 4`

- **Alt sınır:** `Q1 - 1.5 × IQR = 3 - 6 = -3`
- **Üst sınır:** `Q3 + 1.5 × IQR = 7 + 6 = 13`

Çok önemli: -3 ve 13, *aykırı değer tespit sınırlarıdır*; kutu grafiğinin sınırları mutlaka bu sayılarda bitmez. Bu örnekte sınırlar içinde kalan en küçük gözlem 1, en büyük gözlem 8'dir. Dolayısıyla sınırlar **1 ile 8** arasında uzanır; **20** ise sınırın dışında kaldığı için nokta olarak gösterilir.

```
ornek <- c(1, 2, 3, 4, 5, 6, 7, 8, 20)

q1 <- unname(quantile(ornek, 0.25))
q3 <- unname(quantile(ornek, 0.75))
medyan <- median(ornek)

iqr <- q3 - q1

alt_sinir <- q1 - 1.5 * iqr
ust_sinir <- q3 + 1.5 * iqr

# Aykırı değer sınırları içinde kalan gözlemler
sinir_icindeki <- ornek[
  ornek >= alt_sinir & ornek <= ust_sinir
]

# Kutu grafiğinin alt ve üst bıyıkları
alt_cizgi <- min(sinir_icindeki)
ust_cizgi <- max(sinir_icindeki)

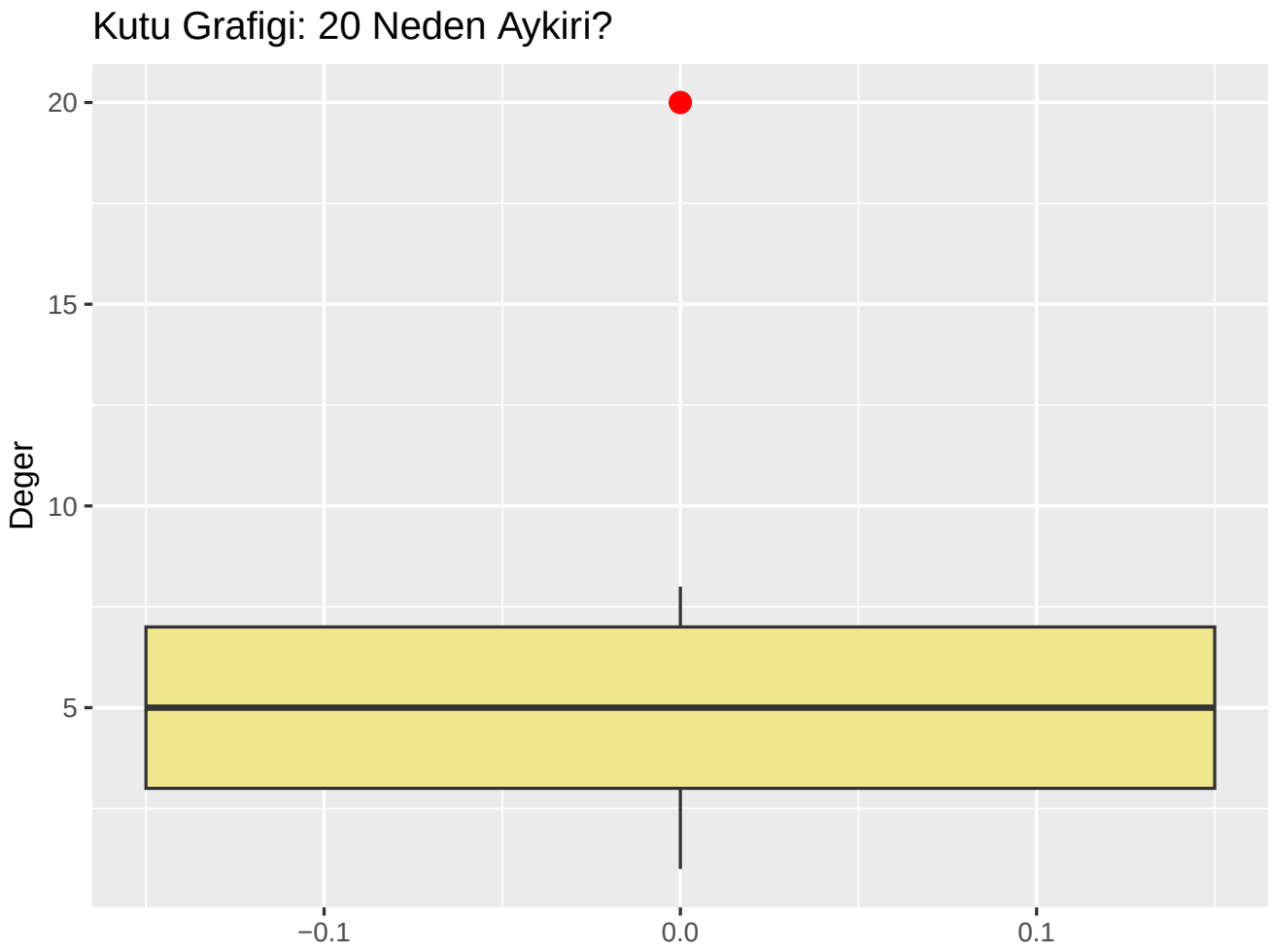
# Olası aykırı değerler
aykiri <- ornek[
  ornek < alt_sinir | ornek > ust_sinir
]

data.frame(
  Q1 = q1,
  Medyan = medyan,
  Q3 = q3,
  IQR = iqr,
  Alt_Sinir = alt_sinir,
  Ust_Sinir = ust_sinir,
  Alt_Cizgi = alt_cizgi,
  Ust_Cizgi = ust_cizgi
)
#>   Q1 Medyan Q3 IQR Alt_Sinir Ust_Sinir Alt_Cizgi Ust_Cizgi
#> 1  3      5  7   4        -3        13         1         8

aykiri
#> [1] 20
```

```
ggplot(data.frame(deger = ornek), aes(y = deger)) +
  geom_boxplot(
    fill = "khaki",
    width = 0.3,
    outlier.color = "red",
    outlier.size = 3
  ) +
  labs(
```

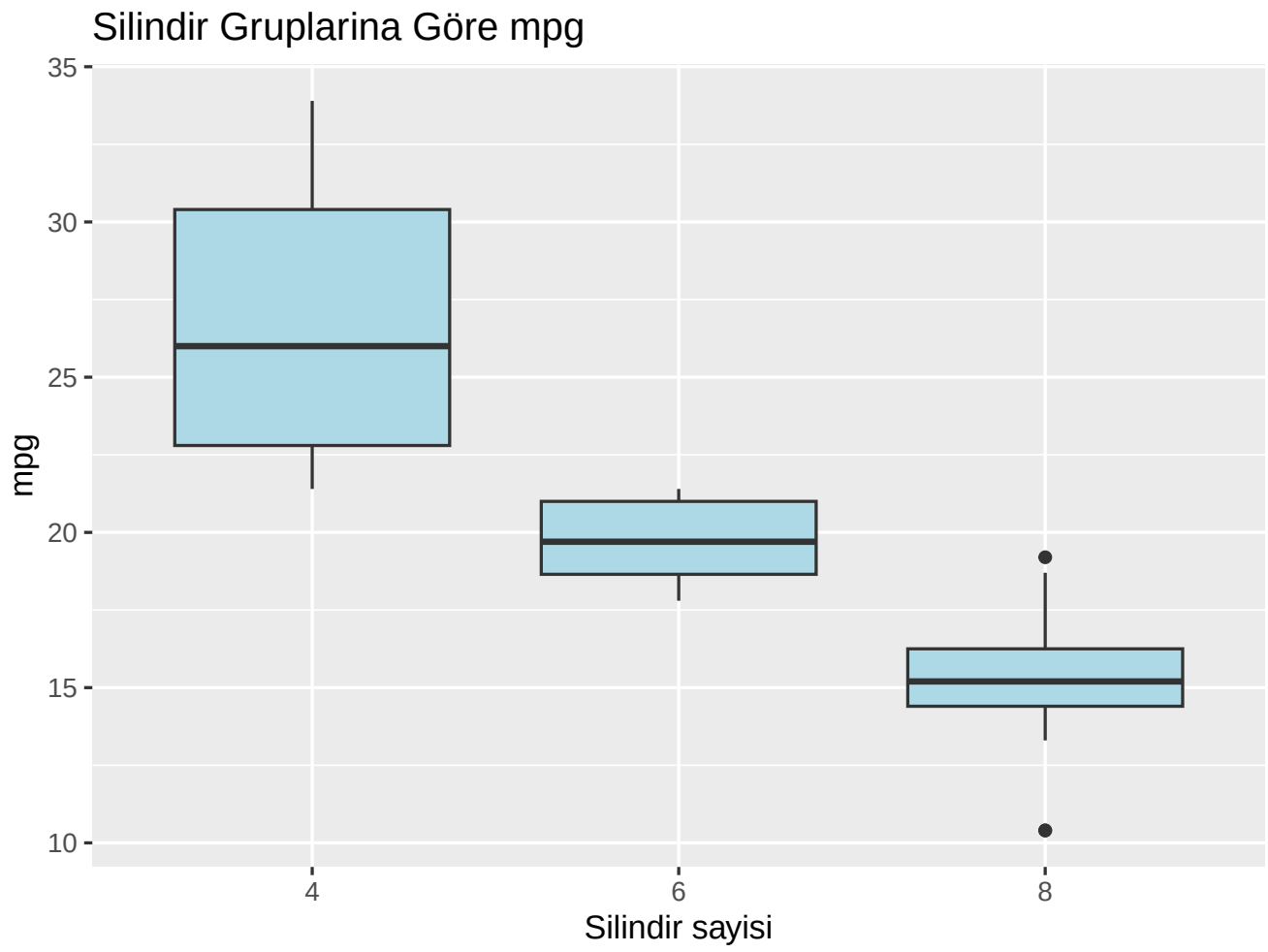
```
    title = "Kutu Grafiği: 20 Neden Aykırı?",
    y = "Değer"
  )
```



Not: `ggplot2::geom_boxplot()` çeyrekleri kendi *hinge* hesaplamasıyla oluşturur. Bu değerler bazı örneklemlerde `quantile()` çıktısından küçük farklar gösterebilir. Yukarıdaki dokuz gözlemli örnekte sonuçlar aynı çıkar.

10.2 9.2. Gerçek veriyle grup karşılaştırması

```
ggplot(df, aes(x = factor(cyl), y = mpg)) +
  geom_boxplot(fill = "lightblue") +
  labs(x = "Silindir sayısı", y = "mpg",
       title = "Silindir Gruplarına Göre mpg")
```




```
# Her gruptaki sayısal özetler
by(df$mpg, df$cyl, summary)
#> df$cyl: 4
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>   21.40   22.80   26.00   26.66   30.40   33.90
#> ------------------------------------------------------------
#> df$cyl: 6
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>   17.80   18.65   19.70   19.74   21.00   21.40
#> ------------------------------------------------------------
#> df$cyl: 8
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>   10.40   14.40   15.20   15.10   16.25   19.20
```

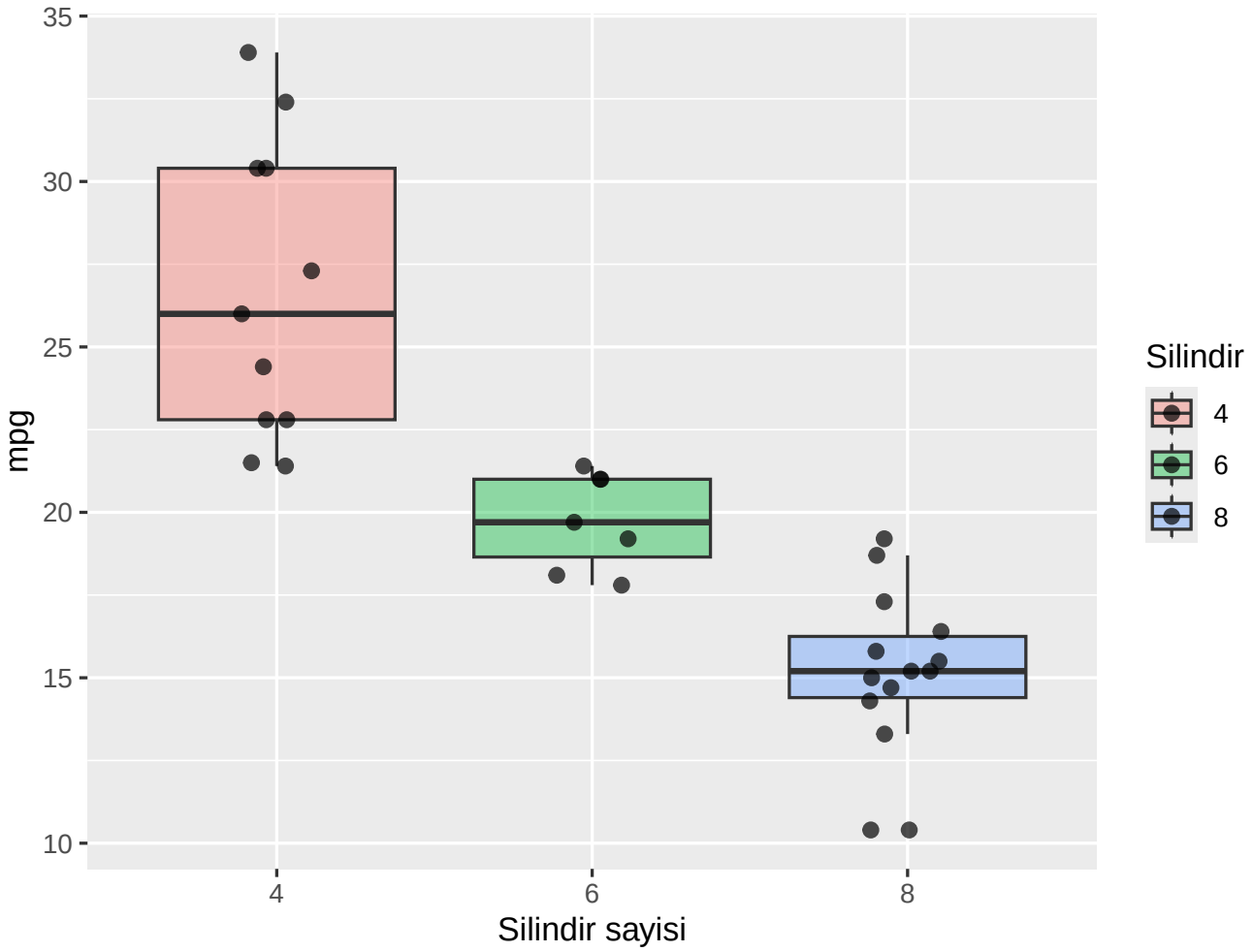
Yorum soruları: Hangi grubun medyanı daha yüksek? Hangi grubun kutusu daha geniş? Aykırı nokta var mı?

10.3 9.3 Kutu Grafiği Üzerine Gözlemleri Eklemek: `geom_jitter()`

Kutu grafiği (`geom_boxplot()`), verilerin medyanını, çeyreklerini, yayılımını ve olası aykırı değerlerini özetler. Ancak her bir gözlemin değerini ayrı ayrı göstermeyebilir.

Veri setindeki gözlemleri de grafik üzerinde görmek istediğimizde `geom_jitter()` fonksiyonunu kullanabiliriz.

```r
ggplot(df, aes(x = factor(cyl), y = mpg,
               fill = factor(cyl))) +
  geom_boxplot(alpha = 0.4, outlier.shape = NA) +
  geom_jitter(width = 0.12, height = 0,
              alpha = 0.7, size = 2) +
  labs(
    x = "Silindir sayısı",
    y = "mpg",
    fill = "Silindir"
  )
```



10.3.0.1 1. `geom_boxplot(alpha = 0.4, outlier.shape = NA)` Bu satır, kutu grafiğini oluşturur.

- **`alpha = 0.4`:** Kutuların saydamlığını belirler. `alpha` değeri 0 ile 1 arasında değişir. Değer 0'a yaklaştıkça grafik öğesi daha saydam, 1'e yaklaştıkça daha opak görünür.
- **`outlier.shape = NA`:** Kutu grafiğinin aykırı değerleri ayrıca nokta olarak çizmesini engeller.

Peki `NA` nedir?

R programında `NA`, genellikle **eksik veya mevcut olmayan bir değeri** ifade eder (*Not Available*).

Ancak bu örnekte `outlier.shape = NA` ifadesi, aykırı değerlerin eksik olduğu anlamına gelmez. `geom_boxplot()` fonksiyonuna aykırı değerler için ayrı bir nokta şekli çizmemesini söyler.

Neden kullanıyoruz?

Çünkü hemen ardından `geom_jitter()` ile bütün gözlemleri nokta olarak göstereceğiz. Kutu grafiğinin aykırı değer noktalarını da ayrıca çizdirirsek aynı gözlem iki kez görünebilir.

Önemli: `outlier.shape = NA` aykırı değerleri veri setinden silmez. Yalnızca kutu grafiği katmanında bu noktaların ayrıca çizilmesini engeller.

10.3.0.2 2. `geom_jitter(width = 0.12, height = 0, alpha = 0.7, size = 2)` `geom_jitter()`, her gözlemi bir nokta olarak gösterir. Noktaların üst üste gelmesini azaltmak amacıyla konumlarına küçük rastgele kaydırmalar uygular.

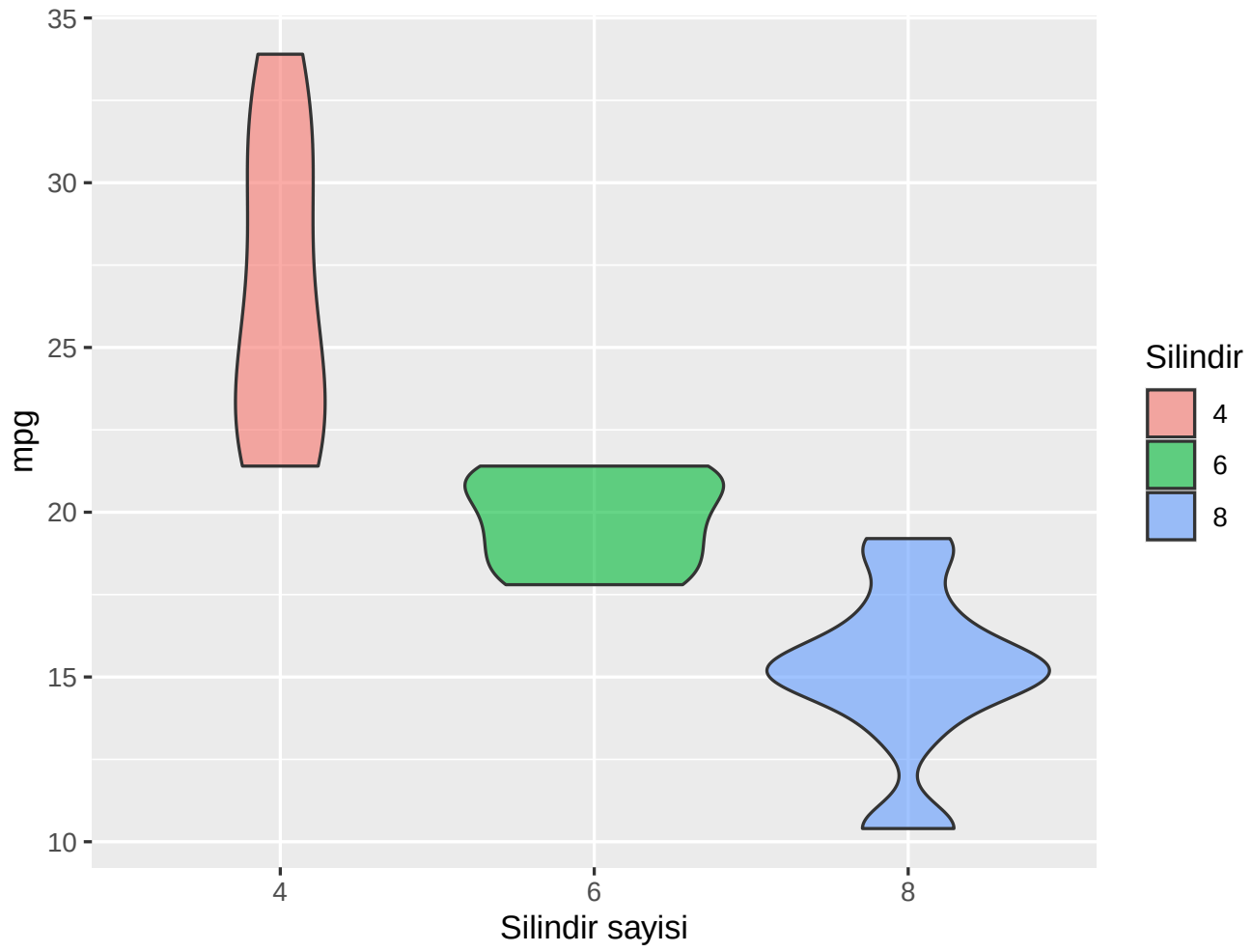
- **`width = 0.12`:** Noktaların yatay eksende sağa veya sola kaydırılabileceği en büyük miktarı belirler. Burada noktalar kendi kategori merkezlerinden en fazla 0.12 birim uzaklaşabilir.
- **`height = 0`:** Noktaların dikey eksende kaydırılmamasını sağlar. Böylece gözlemlerin gerçek `mpg` değerleri korunur.
- **`alpha = 0.7`:** Noktaların saydamlığını belirler. Değer 1'e yaklaştıkça noktalar daha belirgin görünür.
- **`size = 2`:** Noktaların büyüklüğünü belirler.

11 10. `geom_violin()`: Keman Grafiği

Ne zaman kullanılır? Grupların sayısal dağılımlarını yalnızca çeyreklerle değil, tahmini yoğunluk biçimleriyle de karşılaştırmak için.

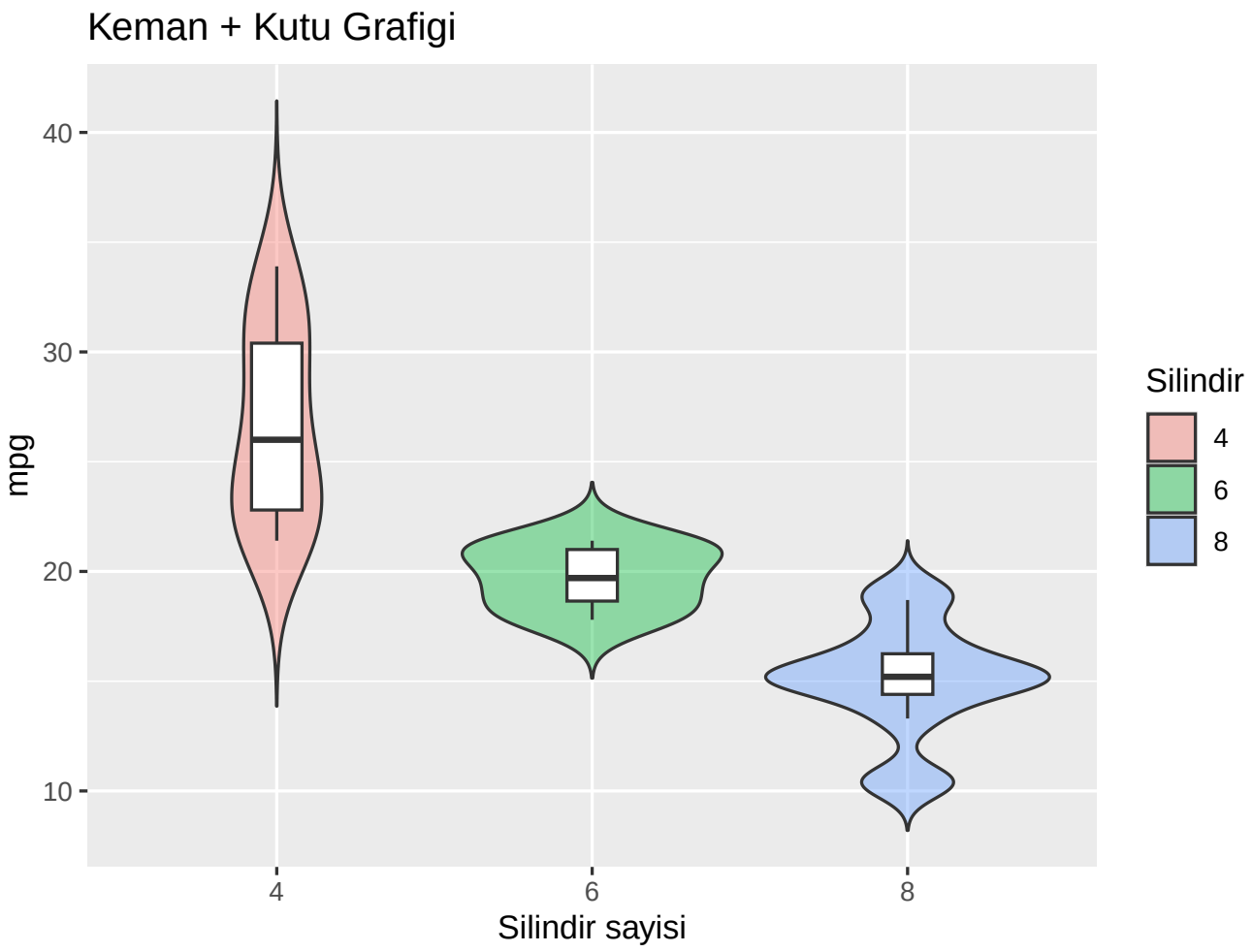
Keman grafiği, iki yana yansıtılmış bir yoğunluk eğrisidir. Geniş bölgeler, ilgili değerler çevresinde tahmini yoğunluğun daha yüksek olduğunu gösterir.

```
ggplot(df, aes(x = factor(cyl), y = mpg,
               fill = factor(cyl))) +
  geom_violin(alpha = 0.6) +
  labs(x = "Silindir sayısı", y = "mpg",
       fill = "Silindir")
```



11.1 10.1. Violin ile boxplot birlikte

```
ggplot(df, aes(x = factor(cyl), y = mpg,
               fill = factor(cyl))) +
  geom_violin(alpha = 0.4, trim = FALSE) +
  geom_boxplot(width = 0.16, fill = "white",
               outlier.shape = NA) +
  labs(x = "Silindir sayısı", y = "mpg",
       title = "Keman + Kutu Grafiği",
       fill = "Silindir")
```



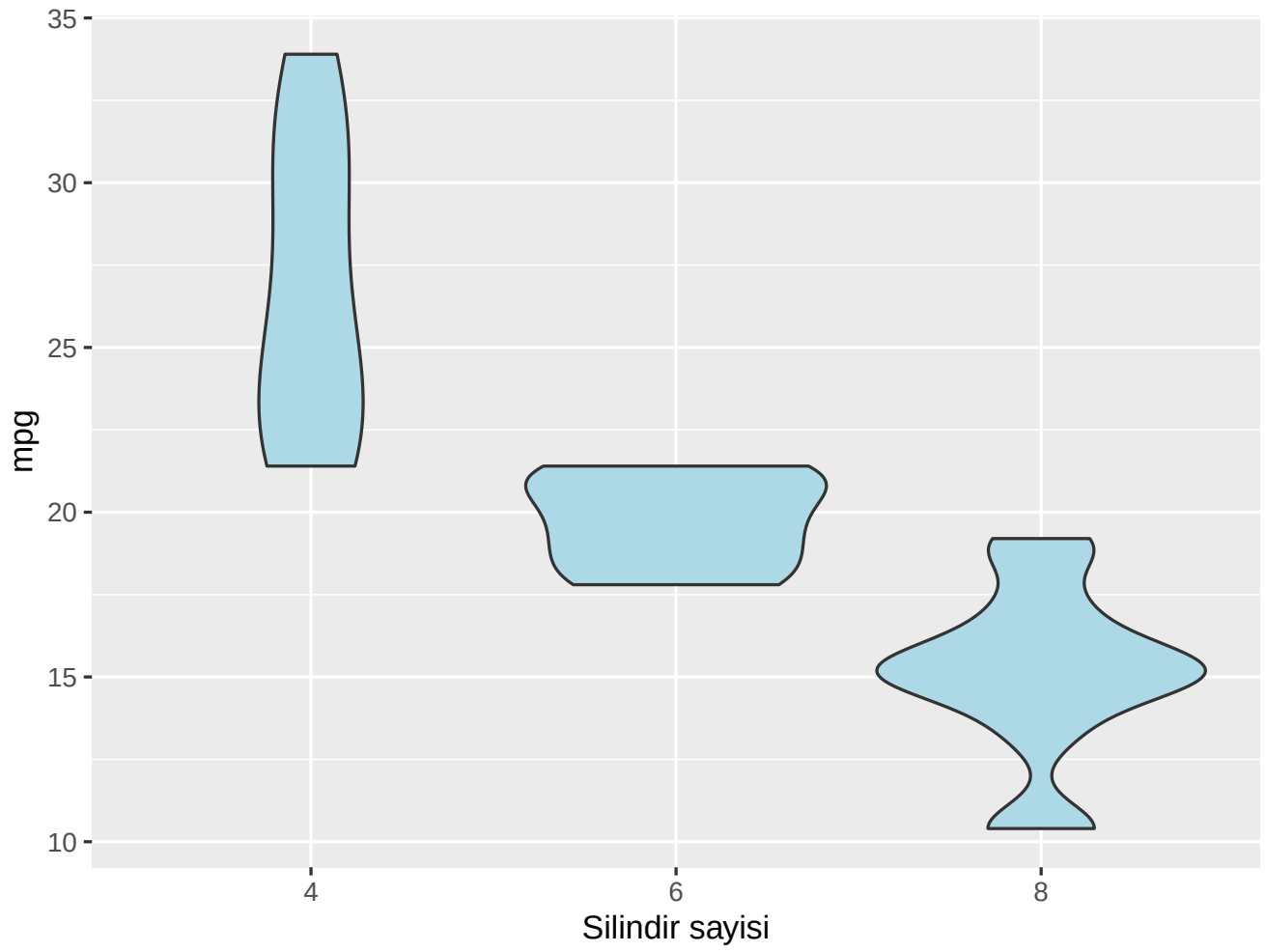
- Keman: dağılımın tahmini biçimini gösterir.
- Kutu: medyanı, çeyrekleri ve bıyıkları gösterir.
- `width = 0.16`: kutunun genişliğini azaltır.

11.1.1 `trim` Argümanı Nedir?

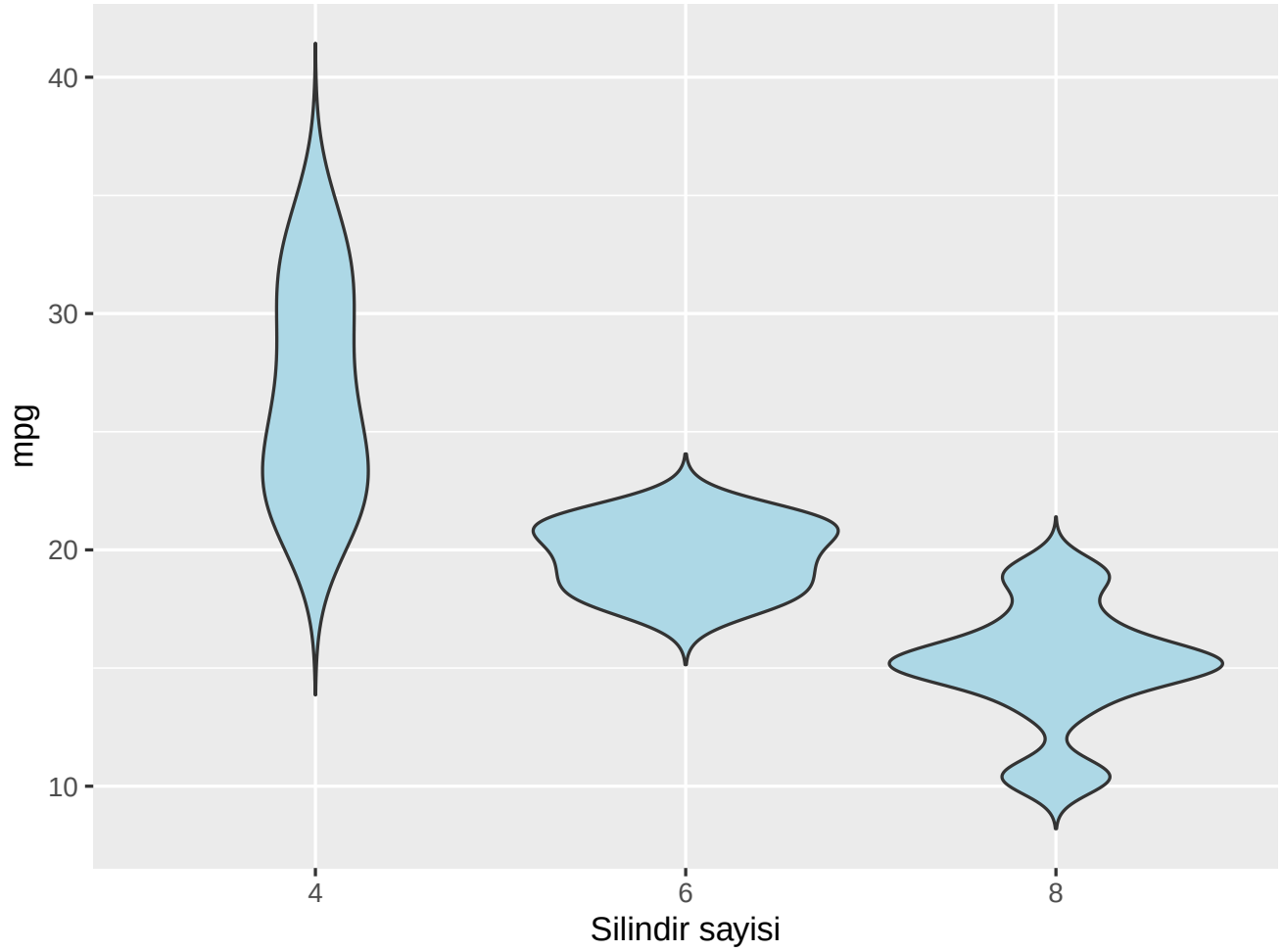
`geom_violin()` fonksiyonunda `trim`, keman grafiğinin uçlarının nerede sonlanacağını belirler.

- **`trim = TRUE`:** Keman grafiğini gözlenen minimum ve maksimum değerlerde sınırlandırır. Varsayılan ayardır.
- **`trim = FALSE`:** Tahmini yoğunluk eğrisinin gözlenen minimum ve maksimum değerlerin ötesine uzanmasına izin verir.

```
# Keman grafiği gözlenen değerlerin sınırlarında kesilir
ggplot(df, aes(x = factor(cyl), y = mpg)) +
  geom_violin(trim = TRUE, fill = "lightblue") +
  labs(x = "Silindir sayısı", y = "mpg")
```




```
# Tahmini yoğunluk eğrisi gözlenen değerlerin dışına uzanabilir
ggplot(df, aes(x = factor(cyl), y = mpg)) +
  geom_violin(trim = FALSE, fill = "lightblue") +
  labs(x = "Silindir sayısı", y = "mpg")
```



Önemli: `trim = FALSE` yeni gözlemler oluşturmaz veya veri setini değiştirmez. Yalnızca tahmini yoğunluk eğrisinin hangi aralıkta çizileceğini etkiler.

Önemli Not: Violin grafiği, gözlemlerin dağılımını bir yoğunluk tahmini kullanarak gösterir. Bu nedenle grafiğin şekli, kullanılan yumuşatma miktarına bağlı olarak değişebilir.

Özellikle az sayıda gözlem içeren gruplarda, grafikte görülen genişlemeler veya birden fazla tepe, verilerde gerçekten farklı alt gruplar bulunduğunu kanıtlamaz.

Bu nedenle violin grafiklerini yorumlarken grup büyüklüklerini dikkate almak ve gerektiğinde gözlemleri nokta olarak da göstermek önemlidir.

12 11. Hangi Grafik Ne Zaman?

Grafik	Veri / amaç	Dikey eksen
`geom_bar()`	Kategorilerin sayısını gösterme	Frekans
`geom_col()`	Önceden hesaplanmış değerleri karşılaştırma	Verilen sayısal değer
`geom_histogram()`	Sayısal değişkenin aralıklara göre dağılımı	Frekans (varsayılan)
`geom_density()`	Sayısal değişkenin yumuşatılmış dağılımı	Yoğunluk

Grafik	Veri / amaç	Dikey eksen
`geom_boxplot()`	Medyan, çeyrekler, yayılım, aykırı değerler	Sayısal değişken
`geom_violin()`	Grupların yoğunluk biçimini karşılaştırma	Sayısal değişken
`geom_point()`	İki sayısal değişkenin ilişkisi	Sayısal değişken

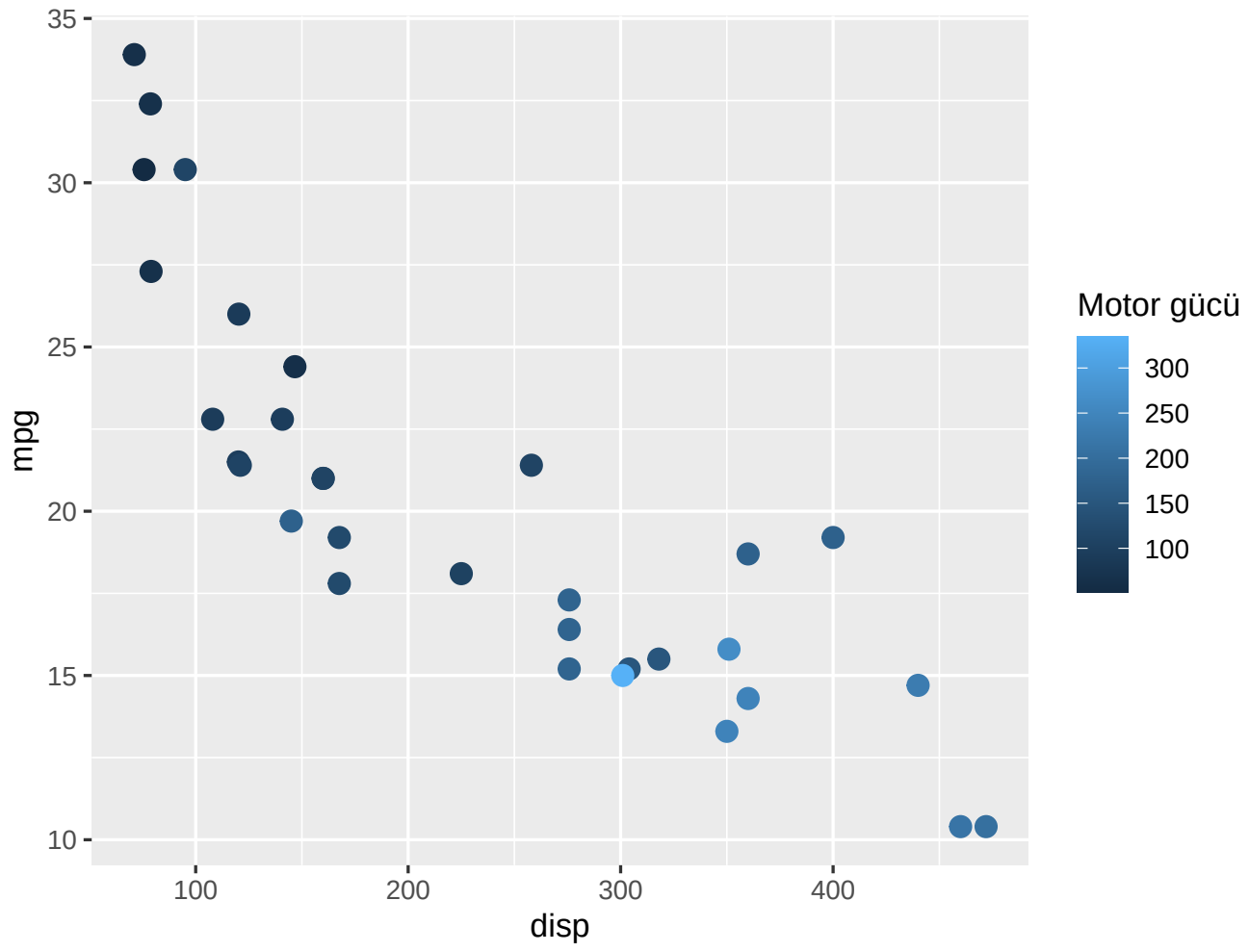
13 Scales (Ölçekler): Veri Değerleri Grafikte Nasıl Gösterilir?

`ggplot2` paketinde **mapping (eşleştirme)** ve **scale (ölçek)** birbiriyle ilişkili ancak farklı kavramlardır.

- **Mapping:** Bir değişkenin hangi görsel özellik ile temsil edileceğini belirler.
- **Scale:** Değişkenin değerlerinin bu görsel özellik üzerinde nasıl gösterileceğini belirler.

Örneğin, motor gücünü (`hp`) noktaların rengine eşleyelim.

```
ggplot(df, aes(x = disp, y = mpg, color = hp)) +
  geom_point(size = 3) +
  labs(color = "Motor gücü")
```



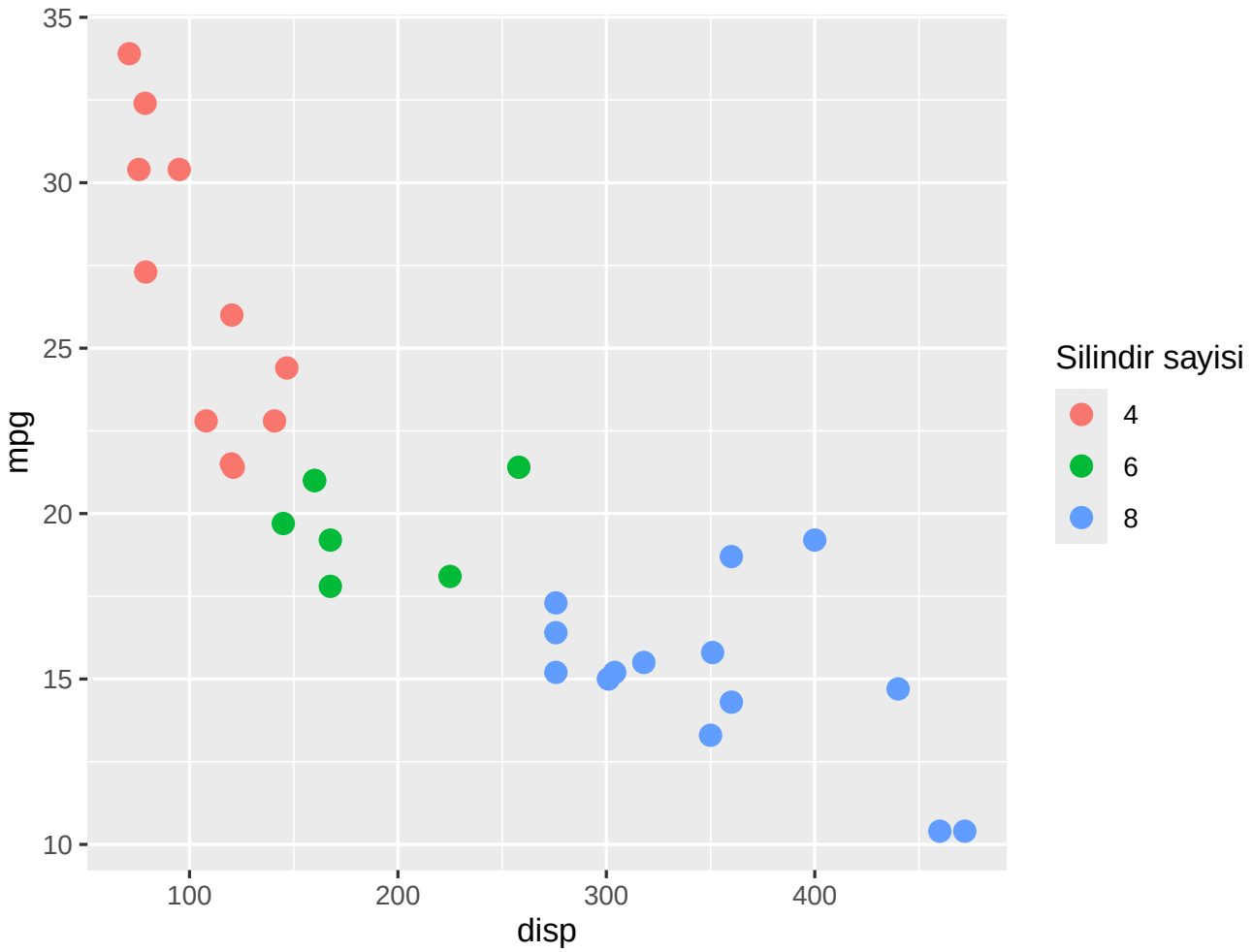
Kodun açıklaması:

- `x = disp`: Motor hacmini yatay eksene yerleştirir.
- `y = mpg`: Yakıt verimliliğini dikey eksene yerleştirir.
- `color = hp`: Motor gücünü noktaların rengine eşler.
- `geom_point(size = 3)`: Noktaları belirli bir büyüklükte çizer.
- `labs(color = "Motor gücü")`: Renk açıklamasının başlığını düzenler.

Burada `hp` sayısal bir değişken olduğu için `ggplot2` otomatik olarak sürekli bir renk ölçeği oluşturur. Farklı motor gücü değerleri farklı renk tonlarıyla gösterilir.

Şimdi silindir sayısını (`cyl`) renge eşleyelim.

```
ggplot(df, aes(x = disp, y = mpg,
               color = factor(cyl))) +
  geom_point(size = 3) +
  labs(color = "Silindir sayısı")
```



Bu örnekte `factor(cyl)` kategorik bir değişken olarak ele alınır. Dolayısıyla `ggplot2`, 4, 6 ve 8 silindirli otomobiller için farklı renkler kullanır.

Önemli: Biz ayrıca bir renk ölçeği tanımlamasak da `ggplot2`, değişkenin türüne uygun varsayılan ölçeği otomatik olarak oluşturur.

Kısacası:

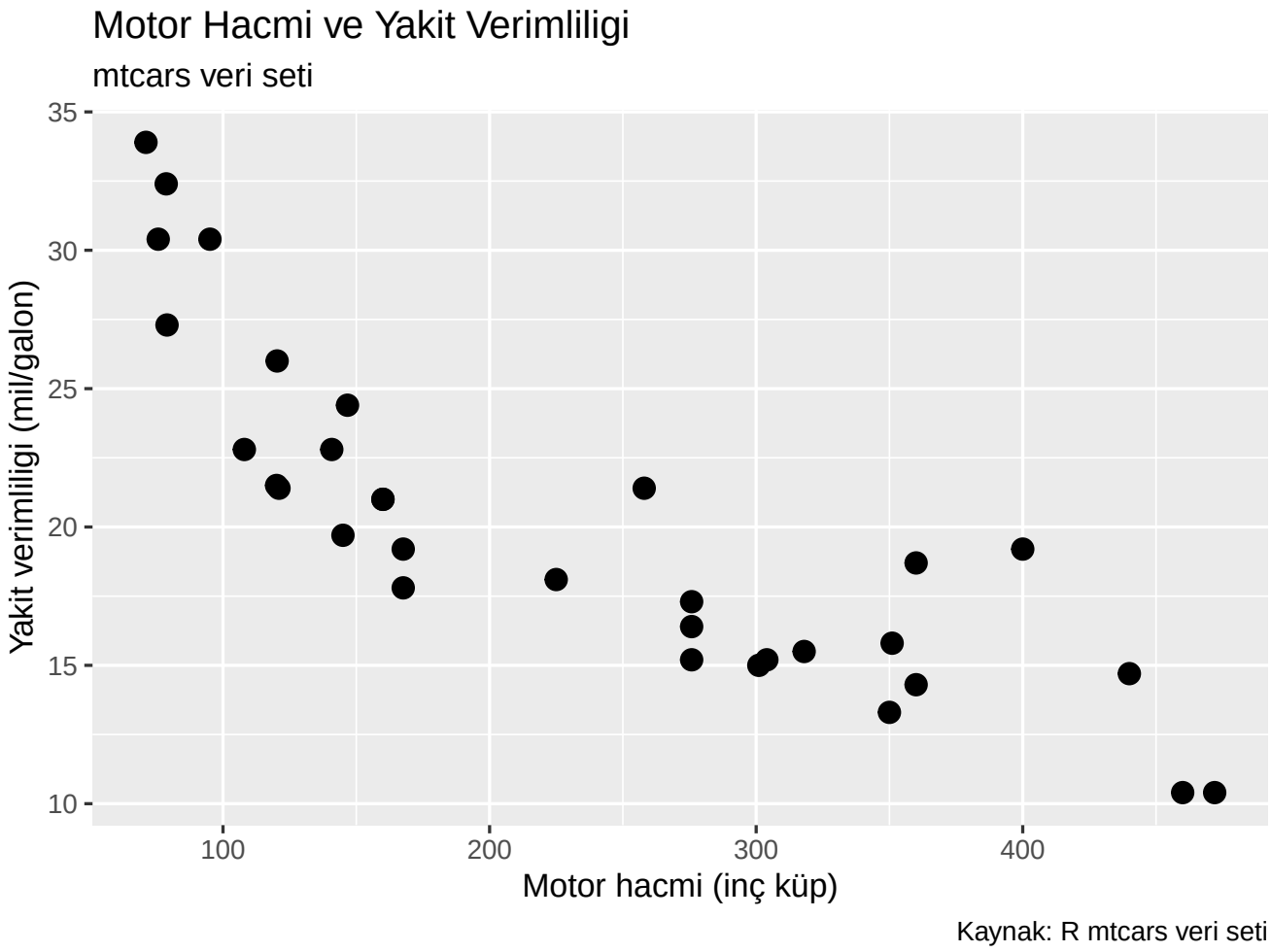
- **Mapping:** "Motor gücünü renkle göstermek istiyorum."

- **Scale:** "Motor gücü değerleri hangi renk tonlarıyla gösterilecek?"

Bu aşamada renkleri `ggplot2` otomatik olarak belirlemektedir.

14 13. `labs()`: Başlık ve Eksenler

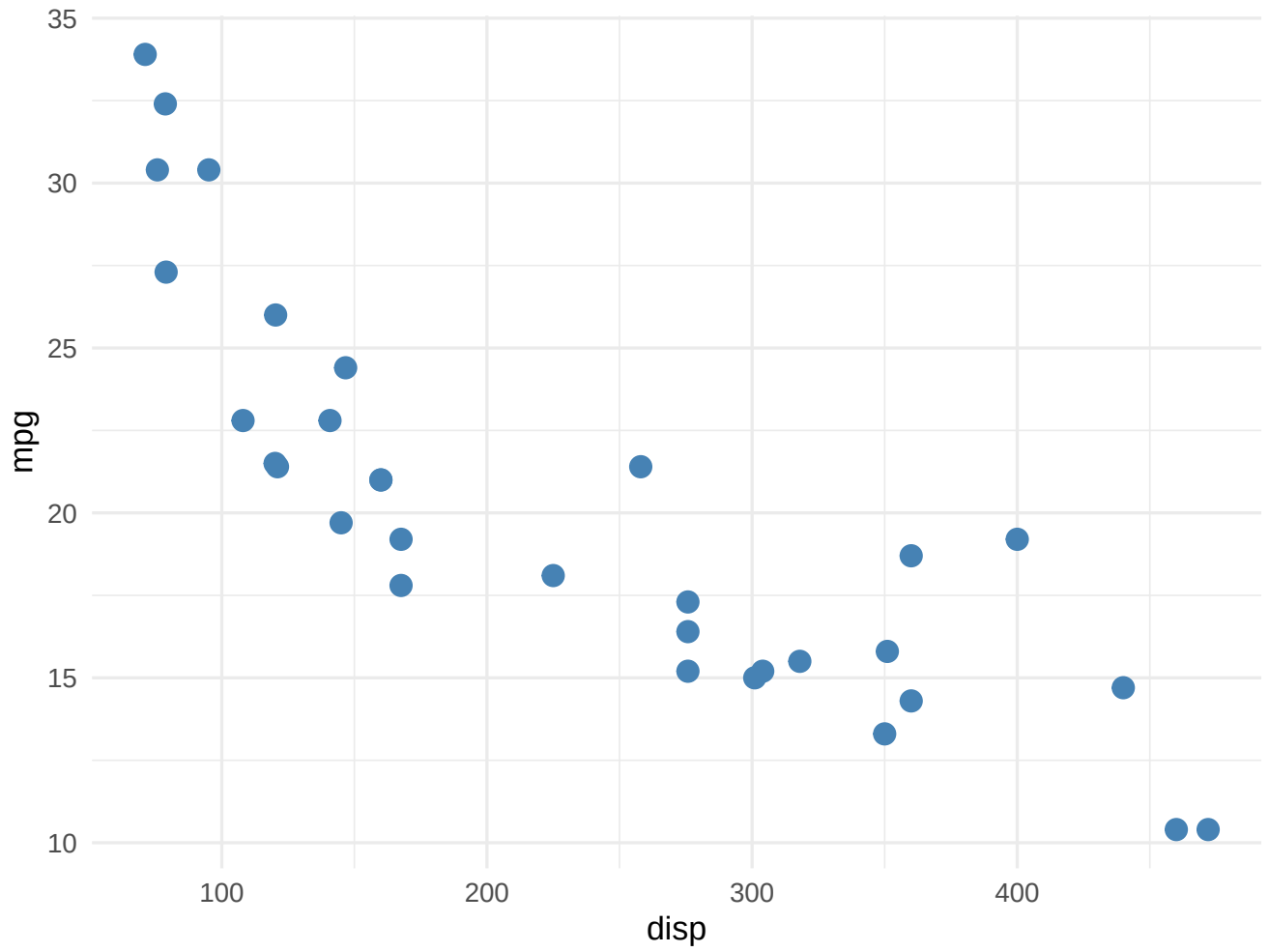
```
ggplot(df, aes(x = disp, y = mpg)) +
  geom_point(size = 3) +
  labs(
    title = "Motor Hacmi ve Yakıt Verimliliği",
    subtitle = "mtcars veri seti",
    x = "Motor hacmi (inç küp)",
    y = "Yakıt verimliliği (mil/galon)",
    caption = "Kaynak: R mtcars veri seti"
  )
```



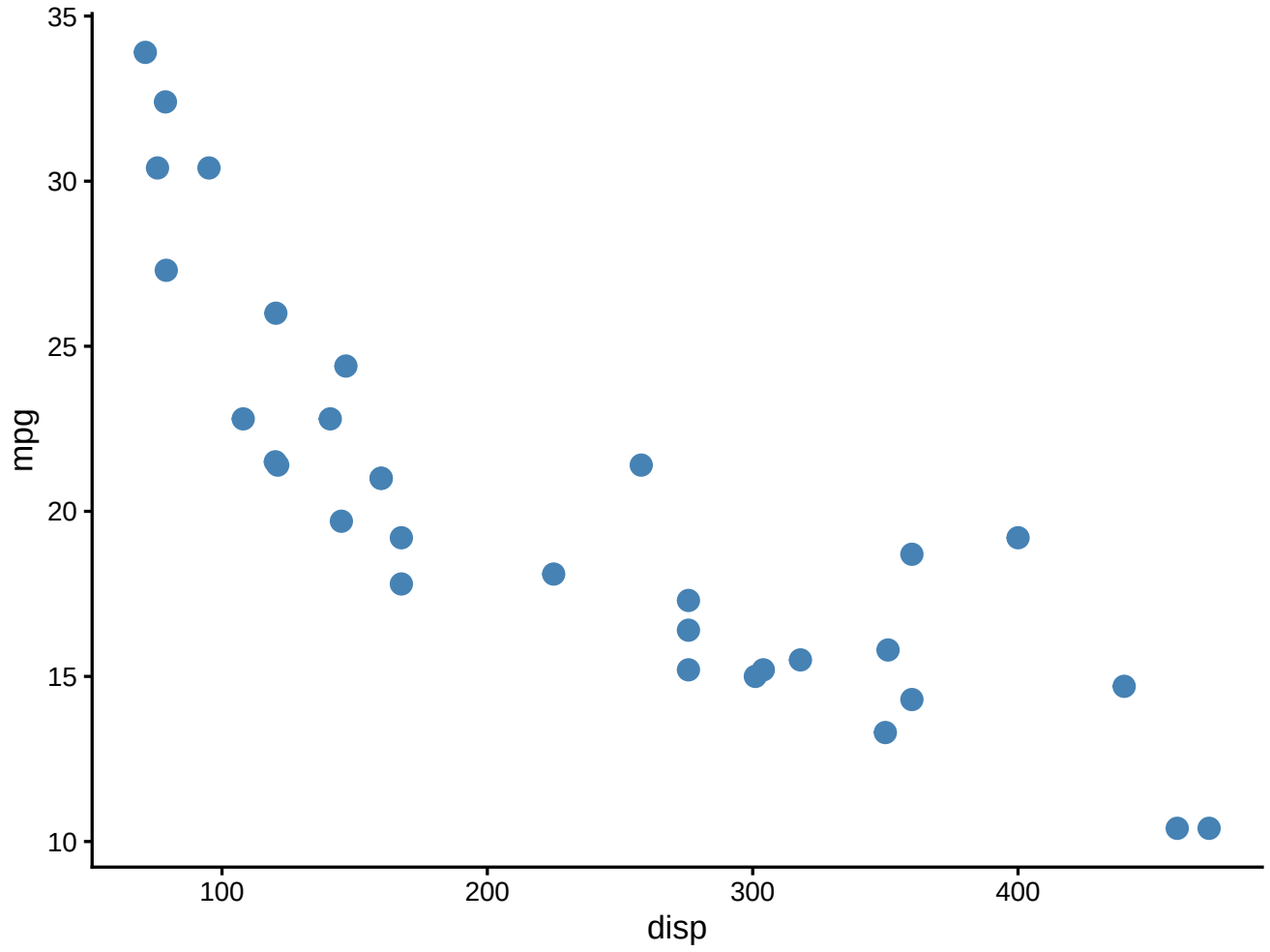
15 14. `theme()`: Grafiğin Görünümü

Tema, verinin eşlenmesini değil; arka plan, yazı, ızgara çizgisi gibi sunum özelliklerini değiştirir.

```r
ggplot(df, aes(x = disp, y = mpg)) +
  geom_point(color = "steelblue", size = 3) +
  theme_minimal()
```



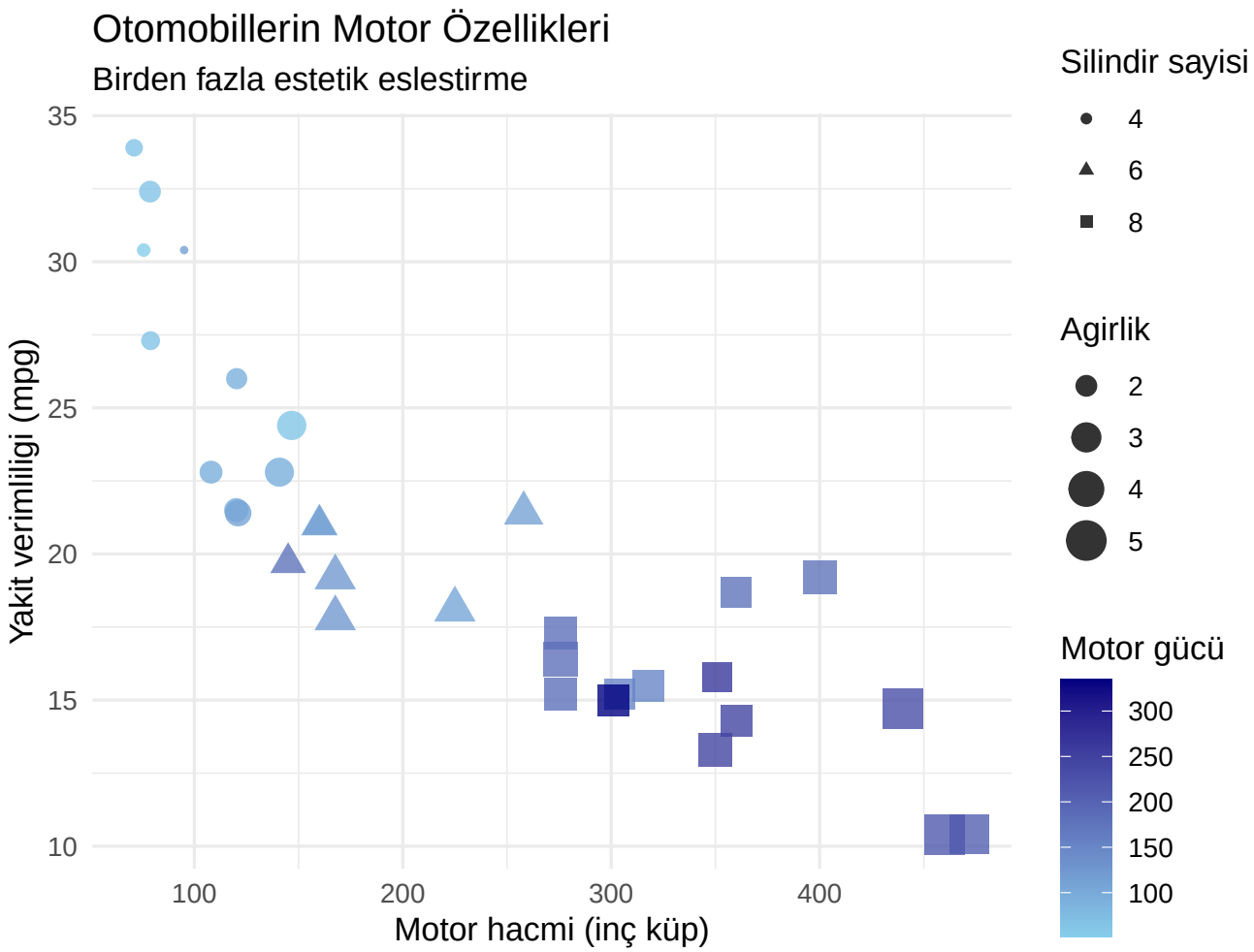

```r
ggplot(df, aes(x = disp, y = mpg)) +
  geom_point(color = "steelblue", size = 3) +
  theme_classic()
```



16 15. Tam Bir ggplot2 Örneği

Şimdi veri, mapping, geometry, scale, labels ve theme katmanlarını birleştirelim.

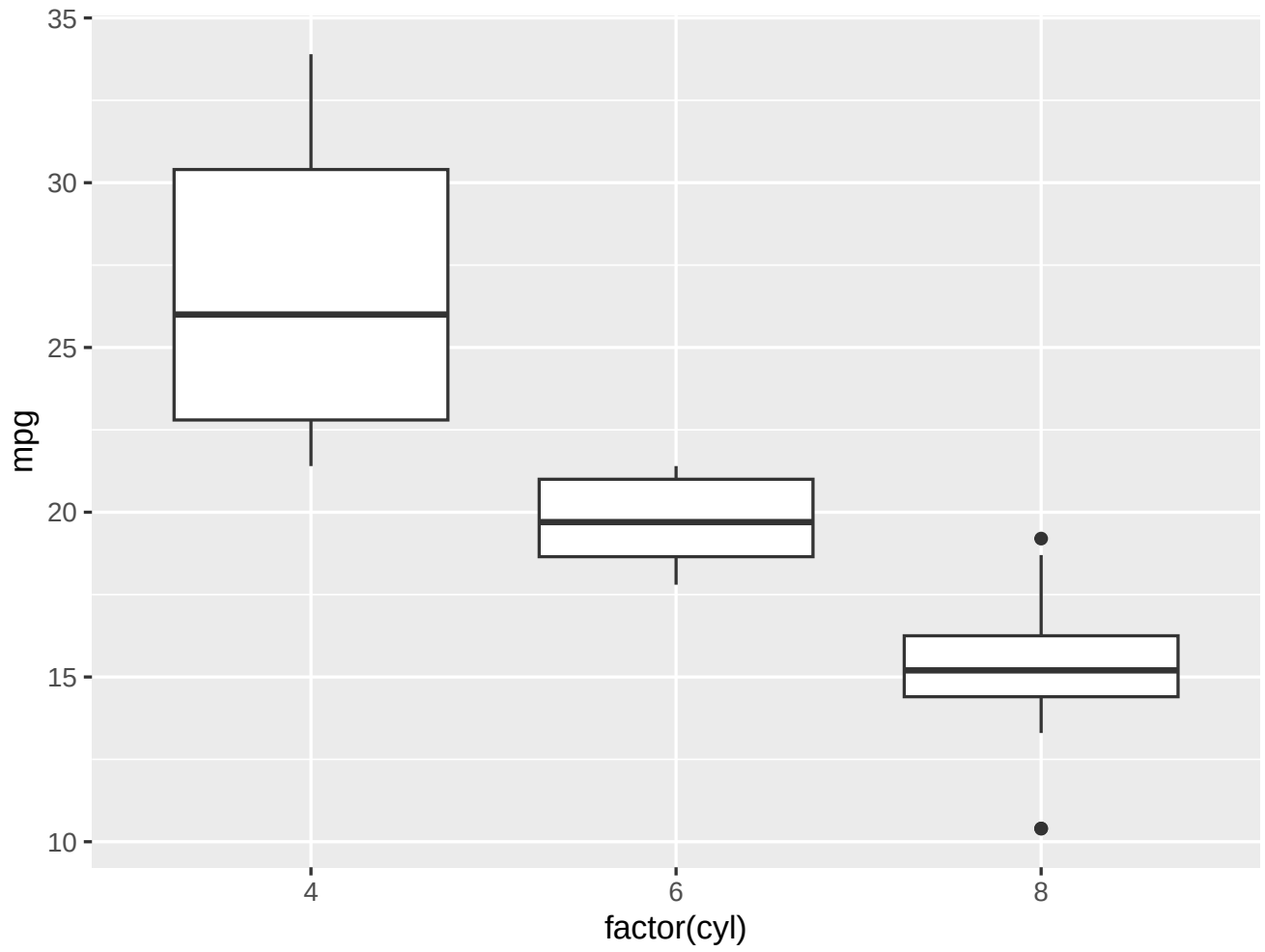
```
ggplot(df, aes(x = disp, y = mpg,
               color = hp,
               size = wt,
               shape = factor(cyl))) +
  geom_point(alpha = 0.8) +
  scale_color_gradient(low = "skyblue", high = "navy") +
  labs(
    title = "Otomobillerin Motor Özellikleri",
    subtitle = "Birden fazla estetik eşleştirme",
    x = "Motor hacmi (inç küp)",
    y = "Yakıt verimliliği (mpg)",
    color = "Motor gücü",
    size = "Ağırlık",
    shape = "Silindir sayısı"
  ) +
  theme_minimal()
```



17 17. Sık Yapılan Hatalar

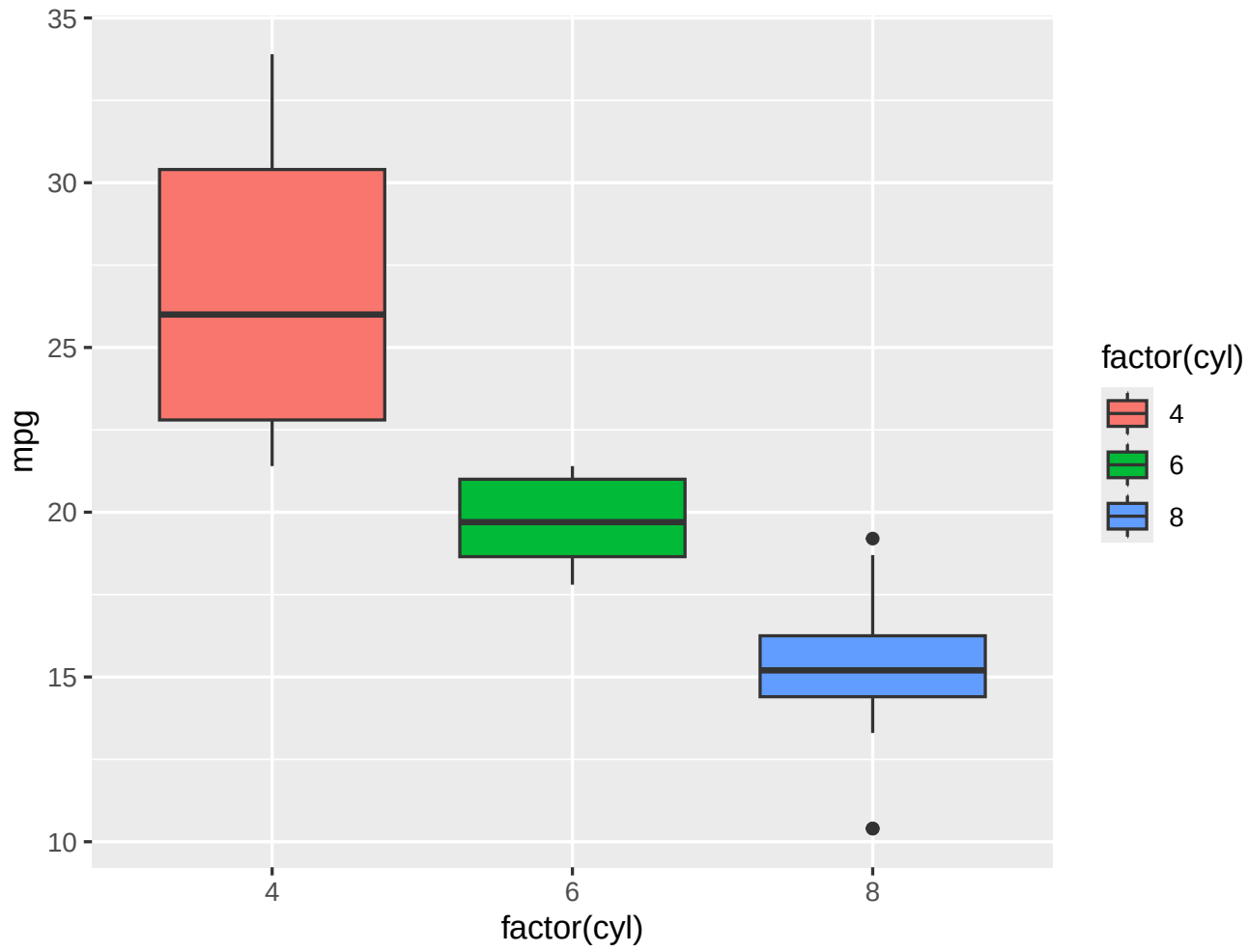
17.1 Hata 1: Sayısal `cyl` ile gruplama beklemek

```
# Grupları açıkça kategorik tanımlıyoruz
ggplot(df, aes(x = factor(cyl), y = mpg)) +
  geom_boxplot()
```



17.2 Hata 2: `fill = cyl` ifadesini `aes()` dışında yazmak

```
# Doğru: dolgu rengini silindir gruplarına bağlamak
ggplot(df, aes(x = factor(cyl), y = mpg,
               fill = factor(cyl))) +
  geom_boxplot()
```



17.3 Hata 3: Histogramda `bins = 5` ifadesini

"her sütunda 5 otomobil var" sanmak

`bins`, aralık sayısını kontrol eder; her aralığın frekansı veriye bağlıdır.

17.4 Hata 4: Boxplot çizgisini teorik sınırla karıştırmak

`Q3 + 1.5 × IQR` üst **sınır** değeridir. Üst bıyık, bu sınırın içinde kalan en büyük gözlemde sonlanır.

17.5 Hata 5: Yoğunluk yüksekliğini frekans sanmak

Yoğunluk eğrisinde toplam **alan** 1'dir; dikey eksen doğrudan kişi/araç sayısı değildir.

18 18. Öğrenci Uygulamaları

Aşağıdaki soruları RStudio'da kendiniz kodlayın.

1. `wt` ve `mpg` için nokta grafiği çizin.
2. Noktaların tamamını `"steelblue"` yapın. Bu mapping mi setting mi?

3. `factor(cyl)` değişkenini renge eşleyin.
4. Silindir gruplarının araç sayısını `geom_bar()` ile gösterin; üstlerine sayıları yazın.
5. `mpg` için `bins = 5` ve `binwidth = 5` histogramlarını karşılaştırın.
6. `mpg` için yoğunluk eğrisi çizin; `adjust = 0.5` ve `adjust = 2` sonuçlarını karşılaştırın.
7. `cyl` gruplarına göre `mpg` kutu grafiği çizin. Medyanları yorumlayın.
8. Aynı gruplar için keman grafiği oluşturun ve kutu grafiğiyle karşılaştırın.
9. Grafiklerden birine başlık, eksen isimleri ve `theme_minimal()` ekleyin.

19 19. Kendinizi Kontrol Edin

1. `ggplot()`, `aes()` ve `geom_point()` hangi görevleri üstlenir?
2. `color = "red"` ile `aes(color = hp)` arasındaki fark nedir?
3. `factor(cyl)` neden kullanılır?
4. `geom_bar()` ile `geom_col()` arasındaki fark nedir?
5. `bins` ve `binwidth` hangi özellikleri değiştirir?
6. Yoğunluk eğrisinde dikey eksen neyi gösterir?
7. Boxplot'ta Q1, medyan, Q3 ve IQR nedir?
8. Üst sınır ile üst çizgi neden her zaman aynı değildir?
9. Violin grafiği boxplot'a hangi bilgiyi ekler?
10. Mapping ve scale arasındaki fark nedir?

20 20. Ders Sonu Özeti

Hatırlanacak temel yapı:

```
ggplot(data = veri, aes(x = degisken1, y = degisken2)) +
  geom_point() +
  labs(title = "Başlık", x = "X ekseni", y = "Y ekseni") +
  theme_minimal()
```

Önce soruyu belirleyin, sonra değişkenleri ve grafik türünü seçin. Bir grafiğin doğru kodlanmış olması, tek başına doğru görselleştirme seçimi yapıldığı anlamına gelmez.